



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**VRLearnTobot Laboratorio
Virtual de programación y
robótica**



Presentado por Luis Ignacio de Luna Gómez
en Universidad de Burgos — 2 de julio de 2025
Tutor: Carlos López Nozal

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	v
Apéndice A Plan de proyecto software	1
A.1. Introducción	1
A.2. Planificación temporal	1
A.3. Estudio de viabilidad	36
Apéndice B Especificación de Requisitos	41
B.1. Introducción	41
B.2. Objetivos generales del proyecto	41
B.3. Catálogo de requisitos	42
B.4. Especificación de requisitos	46
Apéndice C Especificación de Diseño	61
C.1. Introducción	61
C.2. Diseño del sistema de programación inicial (basado en bloques)	62
C.3. Pivote y diseño del sistema híbrido final	64
C.4. Diseño procedimental del sistema híbrido	65
C.5. Diseño arquitectónico	66
C.6. Diseño del sistema de programación híbrido final	72
C.7. Diseño procedimental	73
Apéndice D Documentación Técnica de Programación	87

D.1. Introducción	87
D.2. Estructura de directorios del proyecto primera versión	87
D.3. Estructura de directorios del proyecto segunda versión	91
D.4. Manual del programador (clases y módulos principales)	92
D.5. Compilación, instalación y ejecución del proyecto	95
D.6. Pruebas del sistema	97
Apéndice E Documentación de Usuario	103
E.1. Introducción	103
E.2. Requisitos del sistema y del usuario	103
E.3. Instalación de la aplicación en dispositivo Android	104
E.4. Manual de usuario del prototipo código escrito	105
E.5. Manual de usuario del prototipo programación por bloques .	109
Apéndice F Anexo de sostenibilización curricular	113
F.1. Introducción	113
F.2. Contribución del proyecto a los Objetivos de Desarrollo Sostenible (ODS)	113
F.3. Competencias de sostenibilidad adquiridas y aplicadas	115
F.4. Reflexión final	116
Bibliografía	117

Índice de figuras

A.1. Ejemplo de la estructura de ramas utilizada en el repositorio de GitHub del proyecto, mostrando diferentes líneas de desarrollo para funcionalidades clave.	3
A.2. histograma de commits realizados	3
A.3. Analíticas de espacio de trabajo ofrecidas por Zube para el seguimiento de proyectos.	4
A.4. Ejemplo del tablero Kanban en Zube utilizado para la gestión de tareas de un sprint.	5
A.5. Gráfico Burndown del Sprint 5, ilustrando la reducción de tareas pendientes durante la revisión de las clases MVC.	6
A.6. GráficoBurnup: SP4 - Implantación de la aplicación usando MVC.	6
A.7. Burndown sprint 0	8
A.8. Burndown sprint 1	9
A.9. Burndown sprint 2	11
A.10.Burndown sprint 3	13
A.11.Formato inicial de archivo XML para la configuración de los bloques y su jerarquía.	16
A.12.Burndown sprint 4	17
A.13.Burndown sprint 5	19
A.15.Burndown sprint 6	22
A.14.Depuración de las conexiones desde la consola de unity cuando se ejecutaba el programa.	23
B.1. Diagrama de caso de uso CU - Diagrama caso de uso de la aplicación.	47
B.2. Diagrama de caso de uso CU - Diagrama seleccionar entorno del desafío.	48
B.3. Diagrama de caso de uso CU - Diagrama construir programa.	49

B.4. Diagrama de caso de uso CU03- Ejecutar y simular programa del robot.	56
B.5. Diagrama de caso de uso CU04 - Gestionar proyectos de bloques.	60
C.1. Visión general de la arquitectura MVC del prototipo - primera versión.	66
C.2. Diagrama de la capa modelo.	67
C.3. Diagrama de clases de la capa vista.	68
C.4. Diagrama de clases de la capa controlador.	70
C.5. Diagrama de secuencia: Inicialización interfaz principal.	75
C.6. Diagrama de secuencia: Selección de categoría y visualización de bloques en toolbox.	76
C.7. Diagrama de Secuencia: Inicio del arrastre de un bloque.	77
C.8. Diagrama de clases constructores - factory.	78
C.9. Diagrama de Secuencia: Durante del arrastre de un bloque.	79
C.10. Diagrama de secuencia: Fin arrastre de un bloque.	80
C.11. Diagrama de secuencia: Ejecución de un programa de bloques	82
C.12. Diagrama de secuencia: Flujo de ejecución del programa textual.	84
D.1. Inicialización pruebas en dispositivo Android	99
D.2. Prueba en dispositivo Android	100
D.3. Grabación de la prueba en dispositivo Android	101
D.4. Ejecución exitosa caso de prueba	102
E.1. Vista del menú principal	106
E.2. Vista del menú principal para la selección de desafíos.	106
E.3. Vista general de la Interfaz de Programación (IDE).	107
E.4. Ejemplo de un programa construido con comandos textuales.	108
E.5. Simulación con cámara trasera.	109
E.6. Vista del menú principal bloques de colores.	110
E.7. Vista del menú de desafíos.	111
E.8. Vista de la descripción del desafío.	111
E.9. Vista de la pantalla de programación por bloques.	112
E.10. Vista de la pantalla de simulación.	112

Índice de tablas

B.1. Descripción del Actor A01 - Estudiante.	46
B.2. CU-01: Seleccionar Desafío de Aprendizaje.	48
B.3. CU-02: Construir programa.	49
B.4. CU-02.1: Manipular bloques visuales.	50
B.5. CU-02.2: Añadir comandos textuales.	51
B.6. CU-03: Ejecutar y Probar Programa.	52
B.7. CU03.1 - Iniciar ejecución de programa.	53
B.8. CU03.2 - Observar simulación del robot durante ejecución. . . .	54
B.9. CU03.3 - Detener ejecución de programa.	55
B.10.CU04 - Gestionar proyectos de bloques.	57
B.11.CU04.1 - Guardar proyecto de bloques.	58
B.12.CU04.2 - Cargar proyecto de bloques.	59
C.1. Tabla comparativa de la evolución de componentes arquitectónicos.	85

Apéndice A

Plan de proyecto software

A.1. Introducción

En este plan de proyecto software se detallan los aspectos clave de la planificación, gestión y viabilidad del desarrollo del prototipo del laboratorio virtual de programación y robótica. El presente anexo tiene como objetivo establecer un marco de referencia para la ejecución de este trabajo, asegurando que el producto mínimo viable se desarrolle de manera eficiente y cumpla con los objetivos establecidos.

La planificación se ha enfocado en un desarrollo iterativo, aplicando principios de metodologías ágiles (SCRUM) para dividir el proyecto en tareas, sprints y entregables incrementales. Esto ha permitido una adaptación continua a los desafíos técnicos y una validación temprana de las funcionalidades principales.

En este apéndice, también se incluye un análisis de la viabilidad del proyecto, considerando los recursos humanos y materiales involucrados, los costes asociados estimados para el desarrollo del prototipo y una evaluación básica de los aspectos legales relevantes. El objetivo es demostrar que el proyecto, dentro de su alcance como TFG, es factible y se ha gestionado de manera responsable.

A.2. Planificación temporal

La ejecución de este Trabajo Fin de Grado se ha gestionado mediante una adaptación de la metodología ágil **SCRUM**. Esta elección metodológica

se fundamenta en la necesidad de un desarrollo iterativo e incremental, permitiendo flexibilidad ante los desafíos técnicos y la evolución de los requisitos inherentes a un proyecto de prototipado y desarrollo de software. La planificación se ha estructurado en ciclos de trabajo o *sprints*, de duración variable (generalmente entre una y tres semanas), con el objetivo de producir incrementos funcionales del laboratorio virtual y facilitar una validación continua del progreso.

Herramientas y artefactos de planificación

Para llevar a cabo la planificación temporal y el seguimiento del proyecto, se han utilizado diversas herramientas y artefactos metodológicos.

- **GitHub:** Ha funcionado como el sistema de control de versiones (Git) y el repositorio centralizado para todo el código fuente. Una estrategia fundamental en su uso ha sido la **gestión mediante ramas** (branches) siendo un total de 17 las creadas. Cada nueva funcionalidad significativa o la corrección de errores complejos se abordó en una rama separada ejemplos de ramas utilizadas durante el desarrollo se pueden observar en la Figura A.1).

Este enfoque permitió aislar el desarrollo de estas características, experimentar con soluciones sin afectar la estabilidad de la rama principal (**master**), y facilitó la revisión e integración controlada de los cambios. El historial de *commits* (ver figura A.2) en cada rama ha proporcionado una trazabilidad detallada de la evolución específica de cada módulo o funcionalidad. Adicionalmente, las funcionalidades de *issues* de GitHub se emplearon para el registro y seguimiento de tareas, así como para la gestión de errores o mejoras identificadas durante el desarrollo.

Branch	Updated	Check status	Behind	Ahead	Pull request
CodingProgramming	15 hours ago		6	19	
Scenes	last week		12	0	#100
ProgramacionEstructurada	last week		8	8	
prefabDebugging	2 weeks ago		9	0	#94
Testslintity	27 days ago		61	3	
CodeExecution	last month		61	0	#67
StackSnappingBlocks	last month		73	0	#63
SnappingBlocks	last month		77	0	#60
HightlightConnection	last month		88	0	#58
ConnectionsPVC	2 months ago		93	0	#56
BlockConnections	2 months ago		97	0	#55
BlockJoins	3 months ago		187	1	
connections	3 months ago	0 / 1	198	0	#39
Imagen	3 months ago	1 / 1	198	0	#38
DragAndDrop	3 months ago	1 / 1	203	0	#37
Pruebas	5 months ago		279	0	

Figura A.1: Ejemplo de la estructura de ramas utilizada en el repositorio de GitHub del proyecto, mostrando diferentes líneas de desarrollo para funcionalidades clave.

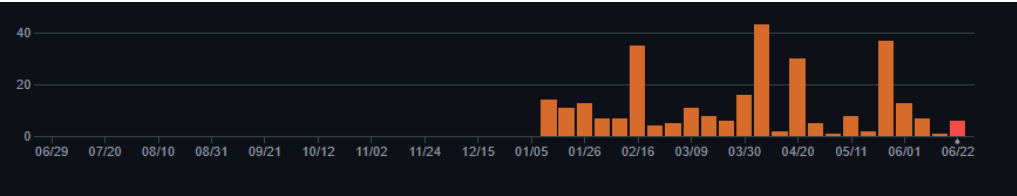


Figura A.2: histograma de commits realizados

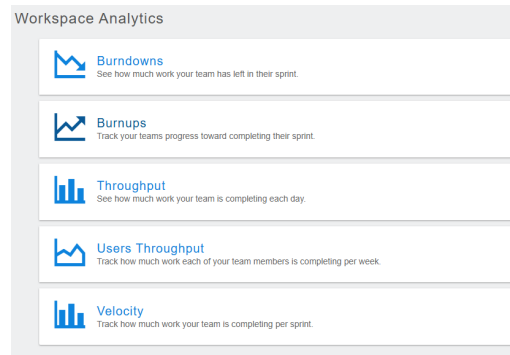


Figura A.3: Analíticas de espacio de trabajo ofrecidas por Zube para el seguimiento de proyectos.

Complementariamente, la plataforma de gestión de proyectos **Zube** [2], integrada con GitHub, fue utilizada para la organización y visualización del trabajo. Sus funcionalidades principales empleadas han sido:

- **Gestión de tareas en tableros Kanban:** Permitiendo la visualización del flujo de trabajo de cada tarea, desde su planificación (Inbox) hasta su finalización (Done), incluyendo estados intermedios. En la Figura A.4 se muestra un ejemplo de este tablero.
- **Planificación y seguimiento de sprints:** Facilitando la definición de objetivos y el conjunto de tareas (historias de usuario con sus respectivos puntos de historia) a abordar en cada iteración.
- **Métricas ágiles y analíticas del espacio de trabajo:** Aunque para un proyecto individual la aplicación estricta de todas las métricas de equipo puede variar, Zube ofrece diversas analíticas (ver Figura A.3) que han servido como referencia para evaluar el progreso general.

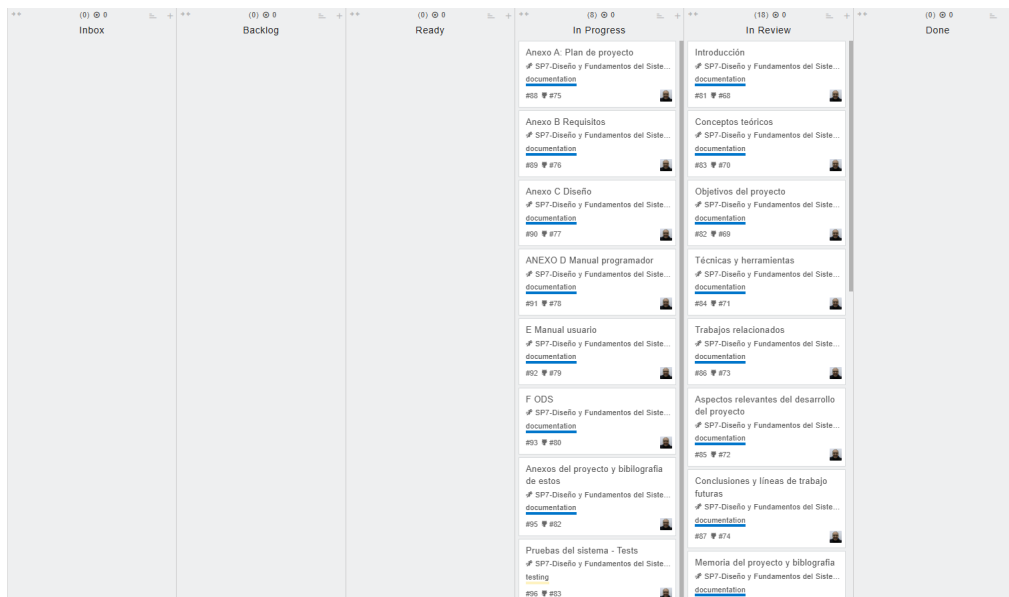


Figura A.4: Ejemplo del tablero Kanban en Zube utilizado para la gestión de tareas de un sprint.

Entre estas métricas, destacan conceptualmente:

- **Gráficos Burndown:** Útiles para visualizar el trabajo restante en un sprint como el que se muestra en la figura A.5.
- **Gráficos Burnup:** Permiten rastrear el progreso acumulado tal y como se puede ver en la figura A.6.
- **Velocity (Velocidad):** Indicador del trabajo completado por sprint, ayudando a calibrar la carga de trabajo futura.

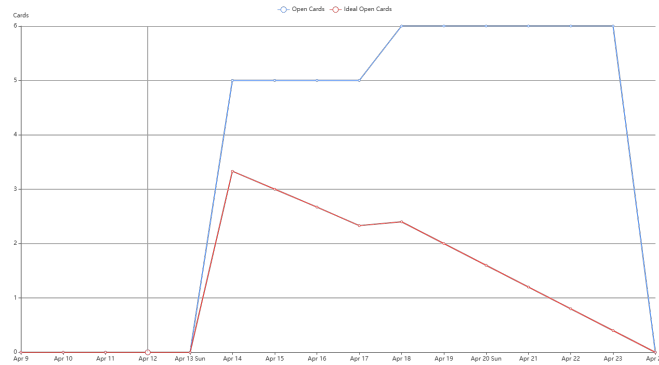


Figura A.5: Gráfico Burndown del Sprint 5, ilustrando la reducción de tareas pendientes durante la revisión de las clases MVC.

La asignación de puntos de historia y su seguimiento en Zube, aunque adaptado a un contexto individual, permitió una estimación cualitativa del esfuerzo y una evaluación continua del avance en las distintas fases del proyecto.

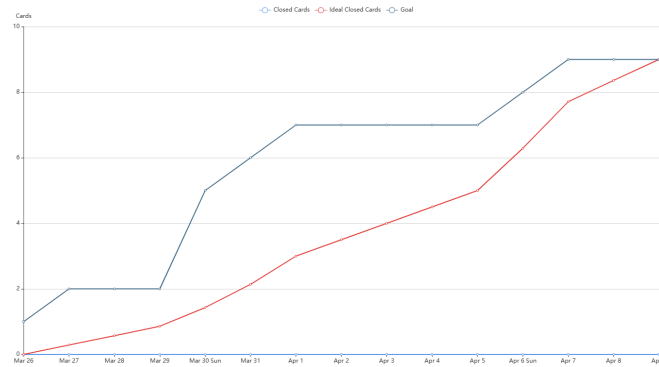


Figura A.6: Gráfico Burnup: SP4 - Implantación de la aplicación usando MVC.

Organización del desarrollo por sprints e hitos clave

El desarrollo del prototipo se articuló en torno a una serie de **hitos principales**, que marcaron la consecución de objetivos clave y agruparon el trabajo realizado a lo largo de varios *sprints*.

Resumen de hitos del proyecto

Los principales hitos que guiaron la evolución del prototipo fueron los siguientes:

- **Hito 1: Fundamentos y arquitectura inicial.** Configuración del entorno de desarrollo, análisis de la librería UBlockly para analizar la posibilidad de reutilizarla en el proyecto.
- **Hito 2: Definición del proyecto y marco teórico.** Consolidación de los objetivos del TFG, estudio comparativo de plataformas de robótica y programación visual, y estructuración inicial de la memoria.
- **Hito 3: Prototipo básico de interfaz y pruebas APK.** Desarrollo de la interfaz de usuario para mostrar paneles y categorías de bloques, y primeras pruebas de generación de APK para Android.
- **Hito 4: Funcionalidad del robot y reestructuración de la simulación.** Superación de las limitaciones de la simulación inicial, desarrollo del RobotBehaviour, implementación de los intérpretes de movimiento y evento, y validación del movimiento básico del robot virtual.
- **Hito 5: Reto técnico y pivote estratégico.** Tras la implementación de las bases, se identificó que la creación de un motor de layout dinámico para los bloques de control de flujo complejos (ej. bucles anidados) era un desafío técnico mayor de lo previsto. En línea con la metodología ágil, tomé la decisión estratégica de pivotar hacia un sistema híbrido. El objetivo era mantener una estructura visual pero introducir un DSL textual para los comandos, asegurando así la viabilidad del proyecto.
- **Hito 6: Desarrollo del sistema híbrido (MVP).** Implementación del CommandParser, el ProgramBuilder y el IDE_UIManager. Adaptación del GameManager y RobotController para ejecutar la List<Instruction> generada por el parser. Diseño de la estructura de ChallengeData para los desafíos.
- **Hito 7: Finalización y pruebas del MVP.** Pruebas integrales del flujo completo (selección de desafío -> escritura de código -> ejecución -> simulación con colisiones). Depuración final y completitud de la memoria del TFG.

A continuación, se ofrece un desglose más detallado del trabajo realizado en cada sprint clave:

Sprint 0: Puesta en marcha, configuración del entorno e investigación inicial

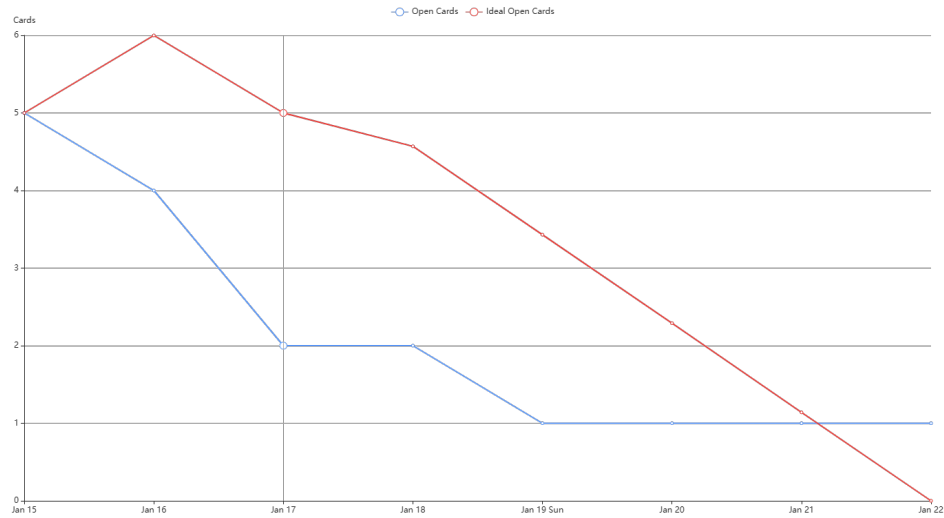


Figura A.7: Burndown sprint 0

- **Duración:** Semana 1 (del 15 al 22 de enero de 2025).
- **Objetivo principal:** Establecer las bases del proyecto, configurar el entorno de desarrollo y realizar una investigación exhaustiva de las herramientas y librerías clave, con especial foco en UBlockly.
- **Contexto:** Tras las reuniones iniciales con el tutor para definir el alcance y los objetivos del TFG, este primer sprint se centró en preparar toda la infraestructura técnica y conceptual necesaria para iniciar la implementación y la configuración de las diferentes partes del repositorio en el que se iba a trabajar.
- **Tareas principales y resultados:**
 - Instalación y configuración de las herramientas de desarrollo: Unity (Versión 6000.0.2f1), C#, Visual Studio, Git y Zube.

- Creación del repositorio del proyecto en GitHub e integración inicial con Zube para la gestión de tareas.
 - Análisis profundo de la arquitectura y componentes del repositorio UBlockly: se estudiaron sus clases **Block**, **Connection**, el sistema de ejecución (**CSharpRunner**, **Cmdtors**) y su integración con Unity UI.
 - Esbozo inicial de la estructura de la interfaz de usuario principal (**UICanvasView**) en Unity, incluyendo la disposición de los paneles para categorías, toolbox y área de codificación.
 - Investigación sobre la representación de robots educativos como LEGO WeDo 2.0 en entornos 3D y los principios de la programación visual estilo Scratch.
- **Hito alcanzado:** Entorno de desarrollo plenamente operativo. Comprensión clara de la base técnica de UBlockly para su adaptación.

Sprint 1: Resumen de objetivos y comparativa de trabajos relacionados

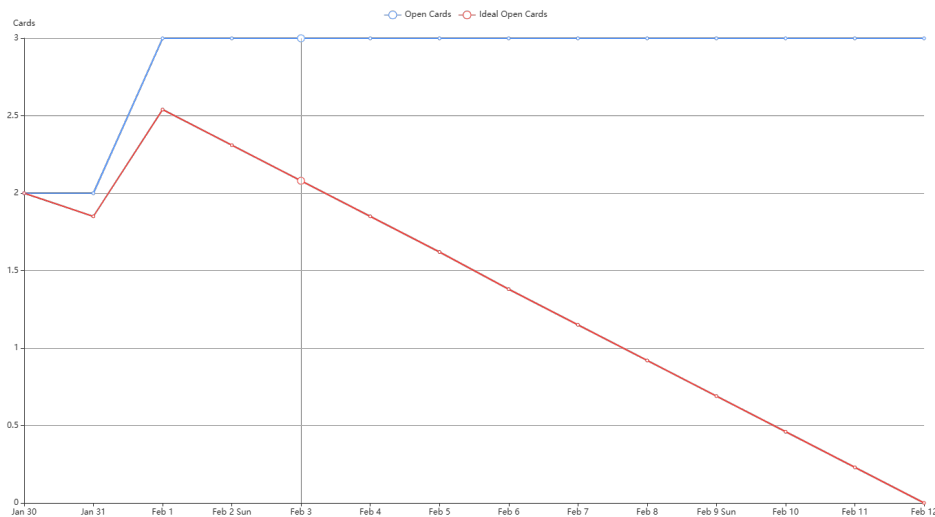


Figura A.8: Burndown sprint 1

- **Duración;** Del 30 de enero de 2025 al 12 de febrero de 2025.

- **Objetivo principal:** Consolidar los objetivos del TFG con un enfoque en la tríada LEGO-Scratch-Unity, realizar un estudio comparativo de robots y paletas de trabajo, y avanzar en la documentación técnica y conceptual.
- **Contexto:** Tras la puesta en marcha inicial del proyecto (Sprint 0), este sprint se centró en definir con mayor precisión el alcance técnico y teórico, así como en establecer una base sólida para la comparativa con trabajos existentes y la estructuración de la memoria.
- **Tareas clave realizadas :**
 - Elaboración de un resumen de los objetivos del proyecto, con un enfoque específico en la integración y adaptación de los paradigmas de LEGO, Scratch y Unity.
 - Creación y refinamiento de la lista de acrónimos a utilizar en la documentación.
 - Sistematización del etiquetado de *issues* y del flujo de *commits* en GitHub para facilitar el seguimiento del tutor.
 - Realización de un estudio comparativo de diferentes kits de robótica educativa y sus entornos de programación para concretar el modelo de robot virtual a simular y sus propiedades fundamentales.
 - Investigación de las paletas de trabajo y tipos de bloques utilizados tanto en Scratch (como estándar de código abierto para la programación visual educativa) como en entornos LEGO, identificando qué estructuras son más adecuadas para el tipo de robot objetivo.
 - Definición detallada de las propiedades del robot virtual que se iban a implementar y simular en el prototipo.
 - Ajuste de la terminología interna del proyecto (ej. cambiar etiqueta formación a learning).
 - Concreción de los objetivos educativos del laboratorio, alineándolos con conceptos de accesibilidad, herramientas de calidad y desarrollo sostenible (ODS).
 - Inicio de la sección de Trabajos Relacionados de la memoria, incluyendo la preparación de una tabla comparativa con las plataformas analizadas.
 - Delimitación conceptual del Laboratorio Virtual frente a un entorno virtual genérico.

- Avance en la documentación contextualizada del proyecto, asegurando que cada concepto teórico estuviera bien anclado y justificado.
 - Documentación de las decisiones de diseño y técnicas tomadas hasta la fecha.
- **Hito alcanzado:** Objetivos del TFG refinados y claramente enfocados. Primera versión del análisis comparativo de plataformas y robots. Estructura inicial de la memoria del TFG definida y con contenido preliminar en secciones clave. Protocolos de gestión de código y tareas establecidos.

Sprint 2: Creación de un prototipo de prueba y avance documental



Figura A.9: Burndown sprint 2

- **Duración:** Del 12 de febrero de 2025 al 5 de marzo de 2025.
- **Objetivo principal:** Desarrollar una primera prueba funcional del robot virtual (inspirado en WeDo 2.0) capaz de moverse en el entorno 3D de Unity, crear un APK para pruebas en dispositivo, y avanzar significativamente en la documentación y estructuración de la memoria del TFG.

- **Contexto:** Sobre la base de los objetivos y la investigación del sprint anterior, este sprint se enfocó en materializar un primer prototipo ejecutable y en consolidar la estructura de la memoria, abordando aspectos técnicos y de contenido.
- **Tareas clave realizadas:**
 - Diseño y desarrollo de una prueba inicial de movimiento para el robot virtual WeDo 2.0 en Unity.
 - Generación de un archivo APK (Android Package Kit) para la instalación y prueba del prototipo en un dispositivo Android físico.
 - Planificación de tareas y organización de *issues* en GitHub, incluyendo su correcta categorización, asignación de salidas esperadas y uso de hashtags para referenciar *commits* asociados.
 - Elaboración de un resumen ejecutivo del trabajo y la descripción del proyecto para la memoria.
 - Refinamiento de la estructura del repositorio, excluyendo archivos innecesarios.
 - Redacción de la sección About (Acerca de) del repositorio del proyecto.
 - Revisión y selección de los Objetivos de Desarrollo Sostenible (ODS 4 y ODS 10) más relevantes para el TFG, justificando su elección.
 - Búsqueda y análisis de artículos y recursos sobre robótica 3D y simulación para enriquecer el marco teórico.
 - Consolidación del formato para las citas bibliográficas (uso de archivos `.bib`, formato autor-año) y referencias.
 - Clarificación conceptual sobre la distinción entre un entorno virtual genérico y la simulación de un robot con comportamiento específico.
 - Revisión y ajuste de los objetivos funcionales y no funcionales del proyecto en base a los avances.
 - Revisión y mejora de la introducción y la sección de objetivos de la memoria del TFG.
 - Definición de los componentes de la prueba en 3D con el robot virtual, incluyendo:

- Modelado 3D básico del robot.
- Diseño de la interfaz de usuario (UI) inicial para la interacción con los bloques y la simulación.
- Elaboración del resumen y título del trabajo para ser incluidos en la memoria.
- Se configuro la herramienta de latex para la generación de los pdfs.
- **Hito alcanzado:** Primera versión de un prototipo de aplicación donde se podía ver los distintos paneles y la proyección de las categorías de los bloques. Avance sustancial en la redacción de la introducción, objetivos, y marco teórico de la memoria. Gestión de tareas y versionado en GitHub/Zube consolidada.

Sprint 3: Reevaluación y desarrollo del prototipo funcional del robot

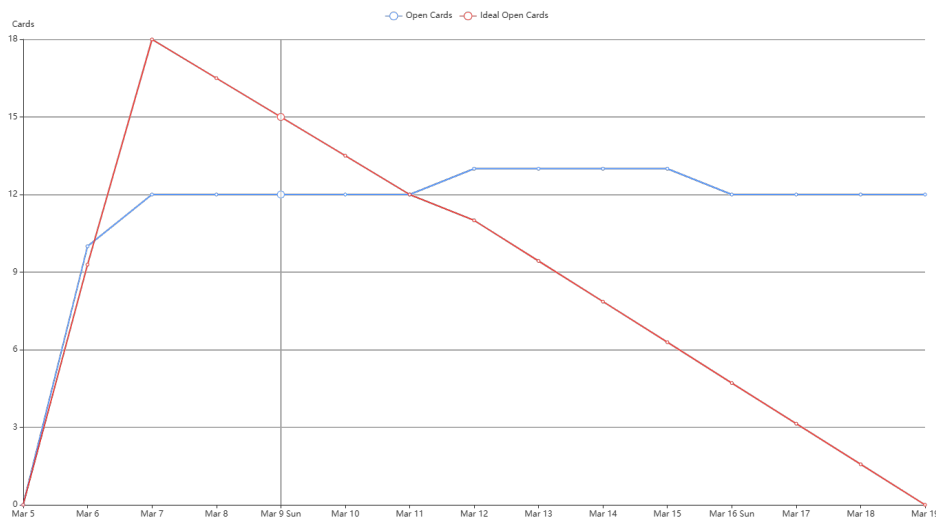


Figura A.10: Burndown sprint 3

- **Duración:** Del 5 de marzo de 2025 al 19 de marzo de 2025.
- **Objetivo principal:** Superar las limitaciones de la simulación básica inicial (basada en la ejecución directa de UBlockly que no permitía una

exportación APK satisfactoria) y desarrollar un prototipo funcional donde el movimiento de un robot virtual 3D fuese visible, controlable a través de los bloques, y empaquetable como una aplicación Android.

- **Contexto y reevaluación:** Las pruebas realizadas en sprints anteriores con la estructura de ejecución directa de UBlockly revelaron que, si bien permitían una lógica de movimiento básica (ej. mover una esfera en el escenario de Unity), no se adaptaban adecuadamente para generar una aplicación Android (.apk) donde un robot 3D claramente identificable ejecutara las acciones programadas por bloques. Esto motivó una **reestructuración del enfoque de la simulación y la interacción**, orientándolo hacia una mayor integración entre la lógica de los bloques de UBlockly (adaptada) y un sistema de control del robot más robusto y visualmente representativo en Unity.
- **Tareas clave realizadas:**
 - **Desarrollo del robot virtual y pruebas de movimiento:**
 - Construcción y/o refinamiento de la versión física del robot con piezas de Lego reales (modelo 3D) para la simulación en Unity.
 - Implementación y depuración de la lógica para la traducción de bloques en una ejecución del movimiento, asegurando que las instrucciones del bloque se convirtieran en acciones concretas del robot virtual
 - Definición y ejecución de la definición prueba en el entorno 3D con el robot virtual para validar su comportamiento básico.
 - **Funcionalidad de bloques (movimiento y eventos):**
 - Desarrollo del Bloque mover - inclusión inputs, asegurando que el bloque de movimiento pudiera aceptar y procesar correctamente los valores de entrada (ej. número de pasos). Se trabajo en entender la estructura de JSON de definición de bloques de UBlockly para adaptarlo al XML con el que se iba a trabajar la configuración de cada tipo de bloque. Se especifican los argumentos que lleva cada bloque (ver [A.11](#)) que permiten definir su jerarquía.
 - Se inicio la implementación de la funcionalidad para Arrastrar y soltar bloques mover y eventos, permitiendo la composición básica de programas visuales.
 - Ajuste del Bloque mover - posicionar las etiquetas para una correcta visualización de sus componentes internos.

- Trabajo en el Bloque eventos - para incluir campos imagen (bandera verde) y posicionar las etiquetas, mejorando la configuración y apariencia del bloque de inicio de eventos.
 - Se empezó a desarrollar de la funcionalidad para soltar bloques en el área de codificación .
 - Se pautaron las bases para la creación y gestión de crear conexiones entre bloques para los bloques de movimiento y evento.
- **Generación y pruebas en dispositivo Android:**
 - Creación y configuración del proyecto para generar un archivo APK (Crear la apk de prueba para su ejecución) para permitir pruebas directas en dispositivos Android.
- **Gestión del contenido visual y documentación:**
 - Ajustar tamaño bloques al contenido para mejorar la presentación visual y el layout dinámico de los bloques según sus campos e inputs .
 - Grabación de video robot real de demostración del prototipo funcional del robot para documentación y seguimiento del progreso.
- **Hito alcanzado:** Se dio forma al prototipo funcional con un robot físico que ejecutaba secuencias de bloques de movimiento iniciadas por un bloque de evento que marcan la base para el prototipo a desarrollar. Se logró la capacidad de generar un APK funcional para pruebas en dispositivo, superando las limitaciones anteriores. Se define además que las versiones deberán ser en 32 y 64 bits por incompatibilidad del Tutor para ponerlo a prueba.

```

<Block>
  <Type>add_to_x</Type>
  <Label>sumar a x %1</Label>
  <SpriteName>Stack_block_grey</SpriteName>
  <connections>
    <top>true</top>
    <bottom>true</bottom>
  </connections>
  <args>
    <arg type="label" value="sumar a x" />
    <arg type="input" name="VALUE" inputType="number" default="5" />
  </args>
</Block>

<Block>
  <Type>subtract_to_x</Type>
  <Label>restar a x %1</Label>
  <SpriteName>Stack_block_grey</SpriteName>
  <connections>
    <top>true</top>
    <bottom>true</bottom>
  </connections>
  <args>
    <arg type="label" value="restar a x" />
    <arg type="input" name="VALUE" inputType="number" default="5" />
  </args>
</Block>

<Block>
  <Type>motion_goto</Type>
  <Label>ir a x:%1 y:%2</Label>
  <SpriteName>Stack_block_grey</SpriteName>
  <connections>
    <top>true</top>
    <bottom>true</bottom>
  </connections>
  <args>
    <arg type="label" value="ir a x:" />
    <arg type="input" name="X" inputType="number" default="0" />
    <arg type="label" value="y:" />
    <arg type="input" name="Y" inputType="number" default="0" />
  </args>
</Block>

```

Figura A.11: Formato inicial de archivo XML para la configuración de los bloques y su jerarquía.

Sprint 4: Reestructuración arquitectónica e implementación del patrón MVC

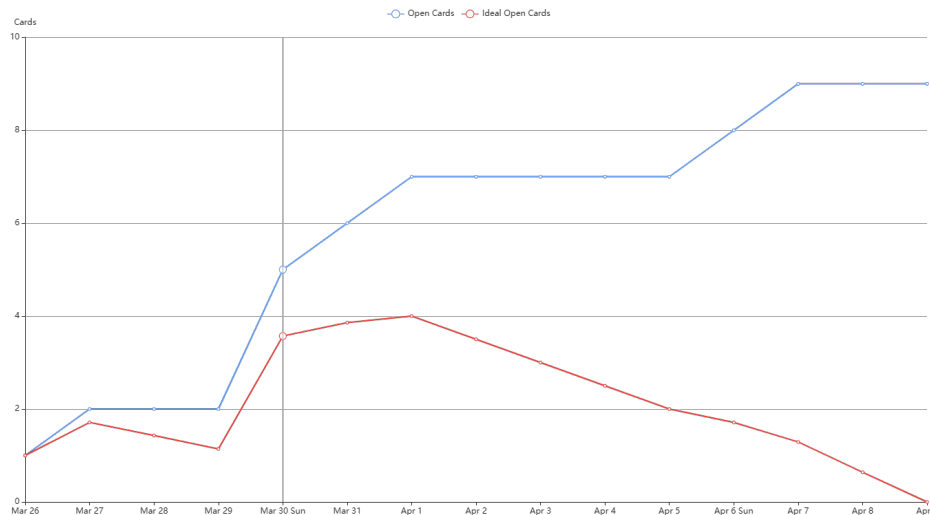


Figura A.12: Burndown sprint 4

- **Duración:** Del 26 de marzo de 2025 al 9 de abril de 2025.
- **Objetivo principal:** Reestructurar la arquitectura del proyecto para implementar de forma explícita el patrón Modelo-Vista-Controlador (MVC). El objetivo era lograr una separación clara de responsabilidades que permitiera modificar o extender componentes del modelo, la vista o los controladores de forma más independiente, mejorando la mantenibilidad y escalabilidad del sistema.
- **Contexto y motivación:** Tras el desarrollo inicial de funcionalidades en sprints anteriores (Sprint 3), se identificó la necesidad de una arquitectura más robusta para gestionar la creciente complejidad, especialmente en la interacción entre la lógica de los bloques (UBlockly), la interfaz de usuario de Unity (Canvas) y la simulación del robot. El patrón MVC fue seleccionado por su probada eficacia en la organización de este tipo de sistemas.
- **Tareas clave realizadas y resultados:**
 - **Análisis y diseño MVC aplicado al proyecto:**

- Identificación de las clases existentes que correspondían a cada capa del patrón MVC (modelo, vista, controlador).
- Diseño de las nuevas clases controlador necesarias para orquestar las interacciones y la lógica de la aplicación.
- **Refactorización y creación de componentes modelo:**
 - Consolidación de las clases que representan el estado y la lógica de negocio pura (ej. `BlockModel`, `ConnectionModel`, `WorkspaceModel` heredados y adaptados de `UBlockly`) como la capa modelo, asegurando su independencia de la interfaz de Unity.
- **Refactorización y creación de componentes vista:**
 - Definición y/o ajuste de las clases `View` (ej. `BlockView`, `InputView`, `UICanvasView`) para que su única responsabilidad fuera la representación visual de los datos del Modelo y la captura de eventos de UI, delegando la lógica a los Controladores.
 - Se revisaron los prefabs de Unity para asegurar que la jerarquía de `GameObjects` reflejara esta separación.
- **Implementación de clases controlador (primera refactorización):**
 - Creación e implementación de los controladores principales, como:
 - ◊ `AppController`: Orquestador general de la aplicación.
 - ◊ `WorkspaceController`: Encargado de las modificaciones al `WorkspaceModel`.
 - ◊ `ExecutionController`: Gestión de la ejecución del programa de bloques.
 - ◊ `InputController`: Manejo de la entrada de datos en los campos de los bloques.
 - ◊ `CategoryController`: Gestión de la selección y muestra de categorías de bloques.
 - Establecimiento de los mecanismos de comunicación entre controladores y las capas de modelo y vista (principalmente mediante el patrón observador y llamadas a métodos específicos).
- **Eliminación de clases y código obsoleto:** Se identificaron y eliminaron clases o fragmentos de código que resultaron redundan-

tes o incompatibles con la nueva arquitectura MVC, simplificando la base del código. .

- **Hito alcanzado:** El proyecto fue transformado exitosamente para operar bajo una arquitectura MVC. Se definieron y crearon los principales controladores, separando la lógica de la presentación y el estado. Esta reestructuración sentó una base mucho más sólida, modular y mantenible para el desarrollo de las funcionalidades subsiguientes y para la futura expansión del laboratorio virtual.

Sprint 5: Revisión profunda de MVC, Interacción de bloques y layout visual

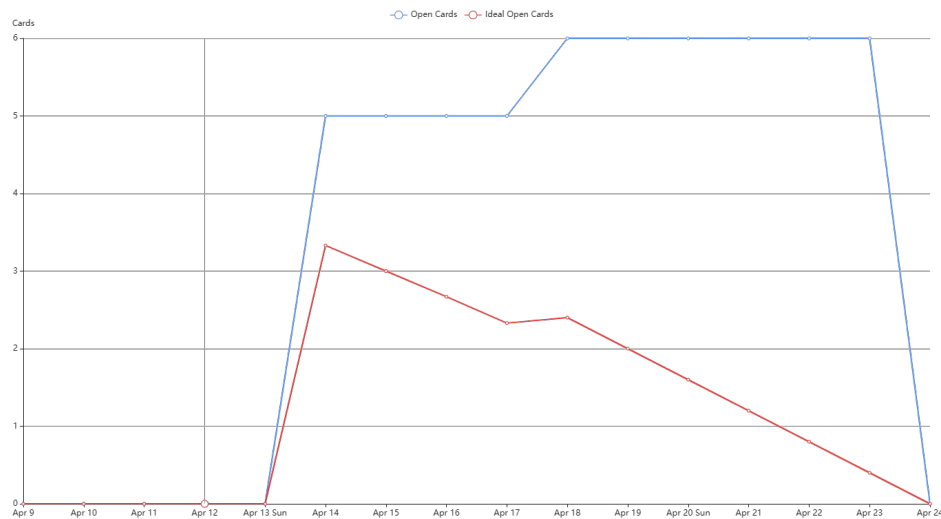


Figura A.13: Burndown sprint 5

- **Duración:** Del 9 de abril de 2025 al 24 de abril de 2025.
- **Objetivo principal:** Refinar la implementación MVC, abordar los complejos desafíos de la detección de conexiones y el posicionamiento de bloques entre diferentes paneles de Unity UI, y comenzar a preparar la escena de simulación del robot.
- **Contexto y motivación:** Con la estructura MVC base implementada en el sprint anterior, este sprint se centró en hacer que la interacción con

los bloques (arrastre, snap, posicionamiento) funcionara de manera robusta y visualmente coherente, enfrentando las particularidades del sistema de coordenadas de Unity y el funcionamiento interno de UBlockly. Se detectó que la interacción entre los distintos paneles de la UI (`Toolbox`, `DragLayer`, `CodingArea`) y sus sistemas de coordenadas relativos eran una fuente importante de complejidad para la lógica de snap y el layout visual.

■ **Tareas Clave realizadas y resultados:**

• **Refinamiento de la interacción Modelo-Vista-Controlador:**

- Revisión y ajuste del código de los intérpretes y generadores de UBlockly para asegurar su correcta adecuación al patrón MVC y su interacción con los nuevos controladores.
- Consolidación de la lógica en los controladores `BlockDragController` y `BlockConnectionController` para la gestión del arrastre de bloques, la detección de posibles conexiones y la interacción con el `WorkspaceModel` a través del `WorkspaceController`.

• **Desafíos en la detección de conexiones y layout:**

- Se inició un trabajo intensivo en la lógica de detección de conexiones (`PreviousConnection`, `NextConnection`, `InputValue`). Este esfuerzo reveló la complejidad de gestionar las posiciones relativas de los bloques cuando se mueven entre paneles con diferentes sistemas de coordenadas (Canvas local vs. Screen space, etc.).
- Desarrollo de un **depurador visual/herramienta de log** para monitorizar en tiempo real las conexiones registradas en la base de datos de conexiones (`BlockConnectionDB`) y el estado de las `ConnectionModel`. Esta herramienta fue crucial para diagnosticar y resolver errores de desajuste, conexiones incorrectas o bloques que no se encontraban para el snap (A.14)

• **Posicionamiento y comportamiento de arrastre de bloques:**

- Se realizaron múltiples iteraciones para asegurar que, al iniciar el arrastre desde la `Toolbox`, el bloque clonado se posicionara correctamente bajo el cursor en la capa de arrastre (`DragLayer`) y siguiera la trayectoria del ratón de forma precisa.

- Se logró que los bloques se depositaran correctamente en el `CodingAreaPanel` al soltar el ratón.
- *Trabajo pendiente identificado*: Se detectó la necesidad de implementar la lógica para eliminar un bloque si, al soltarlo, no aterriza dentro de un área válida (como el `CodingAreaPanel`), replicando el comportamiento de Scratch.
- **Reevaluación de la creación de vistas de bloques (Descubrimiento sobre UBlockly/Prefabs):**
 - Durante el análisis para resolver problemas de layout y conexiones distorsionadas, se descubrió que el enfoque óptimo (y el utilizado por el UBlockly original) es la creación de **prefabs preconfigurados en el editor de Unity** para cada tipo de bloque. Estos prefabs ya contienen la jerarquía correcta de componentes `View` (hijos visuales).
 - Este hallazgo llevó a un **replanteamiento parcial de la estrategia de instanciación de vistas**, enfocándose más en clonar y configurar estos prefabs en lugar de construir la jerarquía visual de cada bloque completamente desde cero en tiempo de ejecución. El objetivo era que la estructura del prefab dictara la disposición interna, evitando errores de posicionamiento derivados de contenedores dinámicos.
- **Problemas y avances en el snap visual:**
 - Se logró la detección lógica de conexiones `NextConnection` y `PreviousConnection` y su vinculación en el modelo.
 - *Desafío persistente identificado*: Al realizar el snap visual, se observaron distorsiones en la imagen del bloque o la destrucción inesperada de la `BlockView`. Se identificó la necesidad de una revisión más profunda del sistema de layout manual (`BaseView.UpdateLayout()`) y cómo interactúa con las propiedades del `RectTransform` al re-parentar y ajustar tamaños tras una conexión.
- **Preparación inicial de la escena de simulación:**
 - Se comenzó a configurar la escena 3D en Unity donde el robot operaría, definiendo el entorno básico y la cámara.
- **Hito alcanzado**: Se logró una mejora significativa en el arrastre y posicionamiento inicial de los bloques en el área de codificación. Se identificaron y comenzaron a abordar los complejos desafíos del snap visual y la correcta gestión de coordenadas entre paneles. El

descubrimiento sobre el uso óptimo de prefabs de UBlockly marcó un punto de inflexión para la estrategia de representación visual. Se dispone de una herramienta de depuración de conexiones. La escena básica para la simulación del robot está iniciada.

Sprint 6: Implementación de interacción Drag & Drop y Pruebas iniciales del prototipo de simulación

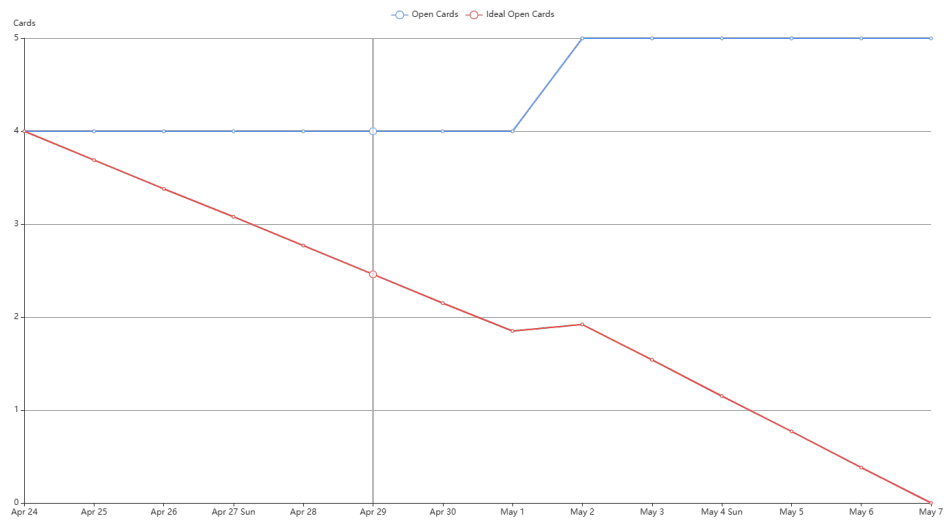
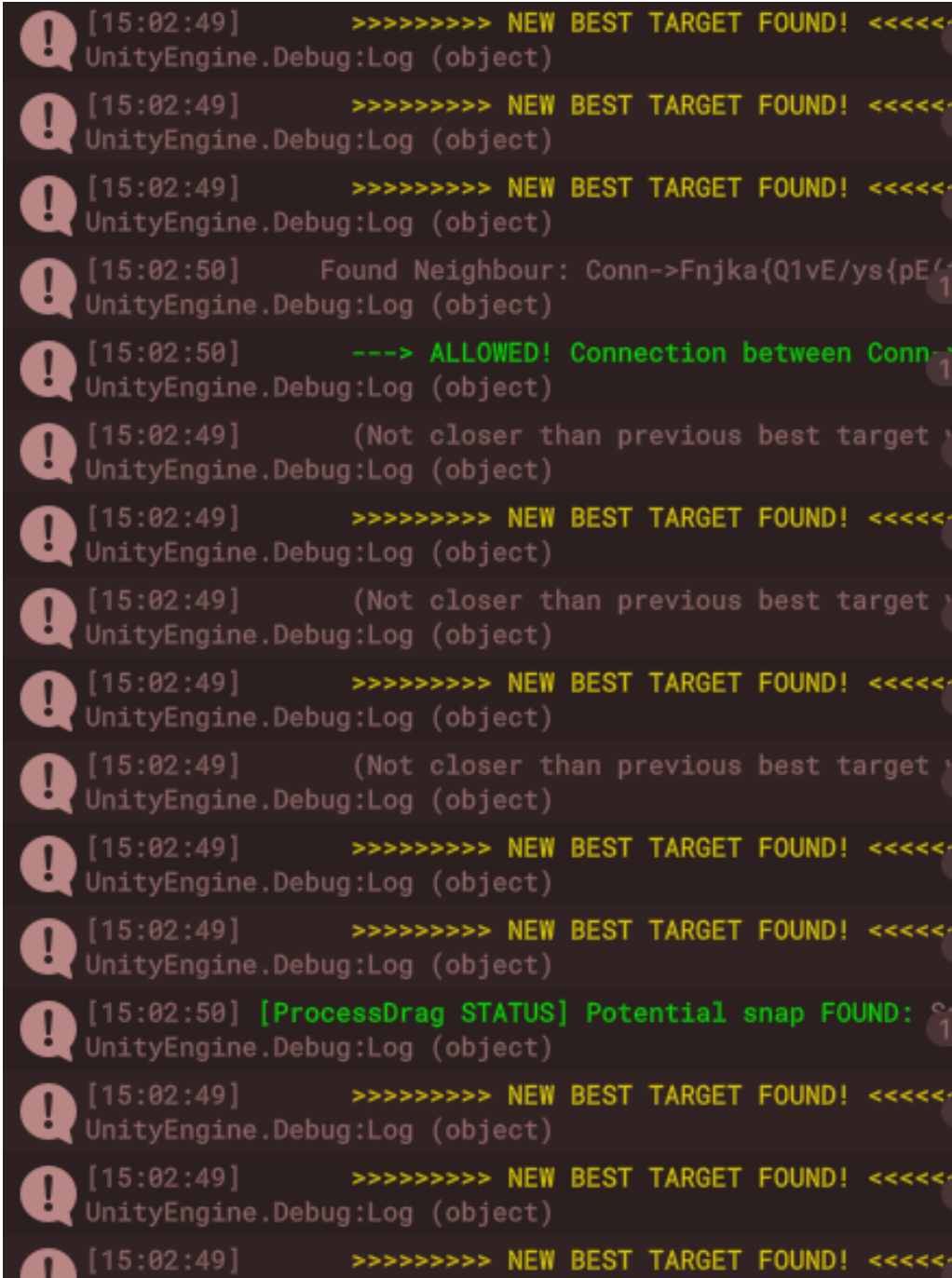


Figura A.15: Burndown sprint 6

- **Duración:** Del 24 de abril de 2025 al 29 de mayo de 2025.
- **Objetivo principal:** Lograr una funcionalidad de arrastrar y soltar (*drag and drop*) robusta para los bloques, incluyendo la conexión (snap) visual y lógica entre ellos. Comenzar las pruebas del prototipo con la secuencia bandera verde seguida de mover 10 pasos para validar el flujo completo hasta la simulación del movimiento del robot.
- **Contexto y desafíos:** Este sprint se enfrentó directamente a los desafíos de layout y posicionamiento identificados en el sprint anterior, particularmente en cómo las vistas de los bloques (**BlockView**) interactúan dentro de los diferentes paneles de Unity UI (Toolbox,

The image shows a screenshot of the Unity console with a dark background and light-colored text. The logs are organized into rows, each starting with a red speech bubble icon containing a white exclamation mark. The text in the logs includes timestamps in square brackets (e.g., [15:02:49]), status messages in all caps (e.g., NEW BEST TARGET FOUND!), and descriptive text in title case (e.g., Found Neighbour: Conn->Fnjka{Q1vE/ys{pE/). Some messages are highlighted in green, such as 'ALLOWED! Connection between Conn-' and '[ProcessDrag STATUS] Potential snap FOUND:'. The logs also include the text 'UnityEngine.Debug:Log (object)' at the end of many entries. The overall layout is a vertical list of these log entries, with some lines truncated on the right side.

```
[15:02:49] >>>>>>> NEW BEST TARGET FOUND! <<<<<<<
UnityEngine.Debug:Log (object)
[15:02:49] >>>>>>> NEW BEST TARGET FOUND! <<<<<<<
UnityEngine.Debug:Log (object)
[15:02:49] >>>>>>> NEW BEST TARGET FOUND! <<<<<<<
UnityEngine.Debug:Log (object)
[15:02:50] Found Neighbour: Conn->Fnjka{Q1vE/ys{pE/
UnityEngine.Debug:Log (object)
[15:02:50] ---> ALLOWED! Connection between Conn-
UnityEngine.Debug:Log (object)
[15:02:49] (Not closer than previous best target
UnityEngine.Debug:Log (object)
[15:02:49] >>>>>>> NEW BEST TARGET FOUND! <<<<<<<
UnityEngine.Debug:Log (object)
[15:02:49] (Not closer than previous best target
UnityEngine.Debug:Log (object)
[15:02:49] >>>>>>> NEW BEST TARGET FOUND! <<<<<<<
UnityEngine.Debug:Log (object)
[15:02:49] (Not closer than previous best target
UnityEngine.Debug:Log (object)
[15:02:49] >>>>>>> NEW BEST TARGET FOUND! <<<<<<<
UnityEngine.Debug:Log (object)
[15:02:49] >>>>>>> NEW BEST TARGET FOUND! <<<<<<<
UnityEngine.Debug:Log (object)
[15:02:50] [ProcessDrag STATUS] Potential snap FOUND:
UnityEngine.Debug:Log (object)
[15:02:49] >>>>>>> NEW BEST TARGET FOUND! <<<<<<<
UnityEngine.Debug:Log (object)
[15:02:49] >>>>>>> NEW BEST TARGET FOUND! <<<<<<<
UnityEngine.Debug:Log (object)
[15:02:49] >>>>>>> NEW BEST TARGET FOUND! <<<<<<<
UnityEngine.Debug:Log (object)
```

Figura A.14: Depuración de las conexiones desde la consola de unity cuando se ejecuta el programa.

DragLayer, Área de Codificación) y sus respectivos sistemas de coordenadas. La correcta implementación del snap visual y lógico era crítica. Adicionalmente, un contratiempo personal (una afección médica ocular provocada por un episodio de alergia intenso) impactó la capacidad de trabajo y extendió la duración de esta fase más allá de lo inicialmente previsto.

■ **Tareas clave realizadas y resultados:**

• **Refinamiento del sistema de conexiones (Drag & Drop):**

- Se revisó y ajustó la lógica para la detección de posiciones cercanas entre los puntos de conexión de los bloques (`Connection_next`, `Connection_prev`, `Connection_output`), utilizando los *prefabs* de bloques diseñados y sus `ConnectionViews`.
- Se abordaron los problemas de conversión entre sistemas de coordenadas globales (pantalla) y locales (Canvas de cada panel) para el correcto posicionamiento y seguimiento de los bloques durante el arrastre.
- Se implementó la funcionalidad de *highlighting* (resaltado visual tipo sombra, similar a Scratch) en la conexión candidata cuando el bloque arrastrado se aproxima a un punto de snap válido. Inicialmente, la sombra no se posicionaba correctamente, pero tras ajustes en cómo se calcula su posición relativa al bloque objetivo, se logró el comportamiento deseado.
- Se logró el snap inicial entre dos bloques (ej. `event_whenflagclicked` y `motion_movesteps`), donde el bloque inferior se unía correctamente al bloque superior.

• **Implementación de apilamiento de bloques (stacking) y lógica de Bumping:**

- Se trabajó en extender la funcionalidad de snap para permitir el apilamiento de múltiples bloques secuenciales (más de dos). Inicialmente, las conexiones `PreviousConnection` y `NextConnection` presentaban problemas al intentar conectar un tercer bloque a una pila ya existente.
- Se identificó y corrigió un error crítico (una referencia nula a la cámara en el sistema *Screen Space Overlay* de Unity UI) que impedía el correcto funcionamiento del snap visual en pilas más largas.

- Se programó e integró la lógica de bumping: cuando un bloque se inserta en medio de una pila ya conectada, los bloques inferiores se desplazan correctamente para hacer espacio, y las conexiones se rehacen de forma adecuada. Se depuró la obtención de la propiedad `Location` en las `ConnectionModel`, crucial para esta funcionalidad.
- Se validó el apilamiento con múltiples bloques `motion_movesteps` bajo un bloque `event_whenflagclicked`.
- **Desarrollo de la simulación y movimiento del robot:**
 - Se configuró la escena 3D con el fondo y el posicionamiento inicial del robot virtual.
 - **Desarrollo del controlador del robot (`RobotBehaviour`):** Se creó (o consolidó) el script `RobotBehaviour` encargado de recibir órdenes de movimiento (ej. mover N unidades) y ejecutar las transformaciones y animaciones correspondientes del `GameObject` del robot en la escena 3D, utilizando corrutinas de Unity para lograr movimientos suaves y controlados en el tiempo.
 - **Implementación del Intérprete de Movimiento y Eventos:** Se desarrollaron y/o adaptaron los intérpretes específicos (`Cmdtors`) para los bloques clave del prototipo:
 - ◊ El `MotionBlockInterpreter` fue implementado para procesar el bloque `motion_movesteps`, extrayendo el parámetro de distancia y traduciéndolo en una orden para el `BlockObserver`.
 - ◊ Se implementó la lógica de interpretación para el bloque `event_whenflagclicked` para que funcionara como punto de inicio de la ejecución del programa.
 - **Integración del sistema de observación y comunicación (`BlockObserver`):** Se refinó el `BlockObserver` para que actuara como el puente crucial entre el sistema de ejecución de bloques (los `Cmdtors`) y el `RobotBehaviour`. Esta clase es responsable de recibir las órdenes lógicas (ej. mover 10 pasos) y delegar la ejecución de la animación/movimiento al `RobotBehaviour`, gestionando además el estado de la simulación.
 - **Pruebas de traducción y ejecución del flujo completo:** Se realizaron verificaciones exhaustivas para asegurar que la secuencia de bloques de evento y movimiento se traducía

correctamente, a través de los intérpretes y el observador, en una secuencia de acciones lógicas y visuales ejecutables por el robot en el entorno 3D.

- Se integraron los botones de bandera verde y stop en la `UICanvasView` para permitir al usuario controlar el inicio y la detención de la ejecución del programa y la transición al modo de simulación.
- Se implementó la lógica de cuenta atrás visual antes de iniciar el movimiento del robot, proporcionando un feedback claro al usuario sobre el inicio inminente de la simulación.
- *Desafío y solución en la simulación del movimiento:* Inicialmente, se observaron problemas donde el robot se movía de forma errática (salía disparado) o no respetaba la secuencia de la cuenta atrás. Se abordó este desafío mediante:
 - ◊ Configuración del `Rigidbody` del robot como `isKinematic = true` para un control programático del movimiento, evitando interferencias no deseadas con el motor de físicas de Unity.
 - ◊ Depuración detallada de la cadena de ejecución de corrutinas entre `CSharpRunner`, `MotionBlockInterpreter`, `BlockObserver`, y `RobotBehaviour` para asegurar que la espera (`yield return`) se propagaba correctamente y que las acciones del robot solo comenzaban tras la finalización de la cuenta atrás y en respuesta directa a las instrucciones de los bloques interpretados.

- **Herramientas de depuración adicionales:**

- Se habilitó la funcionalidad de exportar la base de datos de conexiones (estado del `Workspace`) para facilitar la depuración y el análisis del estado interno del sistema durante las pruebas de conexión.
- **Hito alcanzado:** Se logró una funcionalidad de *drag and drop* y snap robusta para los bloques del prototipo (`event_whenflagclicked` y `motion_movesteps`), incluyendo el apilamiento múltiple y la lógica de bumping. El robot virtual ejecuta los movimientos programados en la escena 3D tras una cuenta atrás, de forma controlada. El flujo principal programar-ejecutar-simular es funcional y demostrable. El sistema permite una exportación del estado de la base de datos de conexiones para depuración.

Sprint 7: Diseño y fundamentos del sistema

Duración: Del 29 de mayo de 2025 al 5 de junio de 2025

Objetivo principal: Producir una primera versión íntegra y de alta calidad de la memoria del Trabajo Fin de Grado, documentando todo el proceso, desde la investigación inicial y la implementación de la arquitectura de bloques.

Contexto y motivación: La primera fase de desarrollo habría concluido. Era fundamental dedicar un sprint completo a la consolidación de la documentación para asegurar que la memoria reflejase de manera precisa, honesta y profesional todo el trabajo realizado hasta este momento, las decisiones tomadas y los aprendizajes obtenidos, preparando la documentación para ser posteriormente revisada en otro sprint para su preparación de cara a una defensa final.

Se procedió a la redacción sistemática de cada uno de los capítulos y apéndices del documento, abordando las siguientes secciones:

Capítulo 1 - Introducción: Desarrollo del marco contextual del proyecto. Se justificó la relevancia de la robótica educativa, se introdujeron los conceptos de programación visual, se expuso la problemática de la brecha digital alineada con los ODS, y se presentó la narrativa del proyecto.

Capítulo 2 - Objetivos del Proyecto: Definición formal de los objetivos, separándolos en funcionales y no funcionales. Los objetivos se redactaron para estar en perfecta consonancia con las características del prototipo hasta este momento desarrollado.

Capítulo 3 - Conceptos Teóricos: Documentación de la base teórica del proyecto. Se incluyó un análisis comparativo de plataformas de referencia (Scratch, Blockly, LEGO). Se detalló la arquitectura inicial investigada, basada en UBlockly, destacando la refactorización a un patrón MVC como aportación propia.

Capítulo 4 - Técnicas y Herramientas: Descripción de la metodología y el stack tecnológico utilizado. Se detalló la aplicación de SCRUM para la gestión del proyecto, el uso de Unity 6 y C#, las herramientas de modelado 3D, y los sistemas de control de versiones y gestión de tareas como Git, GitHub y Zube.

Capítulo 5 - Diseño e Implementación del Prototipo: Este fue el capítulo de mayor desarrollo técnico, donde se detalló la implementación del proyecto en dos fases:

- **Arquitectura Inicial:** Se documentó la implementación del sistema de bloques visuales, la arquitectura MVC diseñada y los complejos sistemas de interacción como el BlockDragController y BlockConnectionController.

Capítulo 6 - Trabajos Relacionados: Se realizó un estudio del estado del arte, comparando el prototipo desarrollado con otras plataformas existentes.

Capítulo 7 - Conclusiones y Líneas de Trabajo Futuras: Se redactaron las conclusiones principales, enfatizando los logros alcanzados y los aprendizajes obtenidos. Se propusieron líneas futuras del trabajo realizado hasta esta fecha.

Bibliografía: Se recopilaron y formatearon todas las referencias bibliográficas utilizadas a lo largo del documento utilizando un gestor de citas compatible con LaTeX, asegurando la rigurosidad académica.

Redacción de apéndices y anexos:

- **Anexo - Plan de proyecto software:** Se documentó de manera detallada todo el plan de gestión del proyecto, incluyendo:
 - La descripción de la metodología SCRUM y las herramientas utilizadas (Zube, GitHub).
 - Un desglose de los hitos principales del proyecto.
 - Una descripción pormenorizada de cada sprint, con sus objetivos, tareas y resultados.
 - Inclusión de artefactos visuales como los gráficos burndown de Zube y capturas de pantalla de los tableros Kanban.
 - Un análisis completo de la viabilidad económica, legal/ética y técnica del proyecto.

Hito Alcanzado: Una primera versión completa y coherente de la memoria del TFG y sus anexos, lista para ser revisada en profundidad por el tutor.

La redacción refleja fielmente la evolución del proyecto, sus contribuciones técnicas y los aprendizajes del proceso. En este sprint se acuerda entre tutor y autor que el producto no es viable para ser presentado en una primera defensa y se establecen nuevas líneas de trabajo para mejorar lo existente.

Sprint 8: Introducción guiada y desafíos de programación

Duración: Del 5 de junio de 2025 al 12 de junio de 2025

Objetivo principal: Transformar el prototipo de un simple sandbox de programación en una herramienta educativa estructurada, mediante el diseño e implementación de un sistema de desafíos guiados. El objetivo era crear un flujo de usuario completo, desde un menú principal hasta la resolución de un problema concreto en el entorno de simulación.

Contexto y motivación: Con un motor de simulación funcional donde el robot se movía y giraba de forma predecible, el siguiente paso lógico era darle un propósito educativo a estas acciones. Se identificó que, para maximizar el aprendizaje y la motivación del usuario, era necesario contextualizar la programación dentro de objetivos específicos y medibles. Este sprint representó un punto de inflexión clave, desplazando el foco desde la pura mecánica de la programación (visual o textual) hacia la aplicación de la lógica para la resolución de problemas.

Tareas clave realizadas y resultados:

■ Consolidación de la simulación base:

- Se estabilizó y refinó el comportamiento del `RobotController`, asegurando que el movimiento (mover X pasos) y el giro (girar Y grados) fueran consistentes y predecibles.
- Se implementó un sistema multicámara (`CameraManager`) en la escena de simulación, permitiendo al usuario cambiar la perspectiva para observar la acción del robot desde diferentes ángulos.

■ Diseño del sistema de gestión de desafíos:

- Se tomó la decisión de utilizar `Scriptable Objects` para definir la estructura de los desafíos (`ChallengeData`). Este enfoque

de Unity permite encapsular los datos de cada desafío (título, descripción detallada, etc.) como activos independientes en el proyecto, facilitando enormemente la creación, gestión y expansión del contenido educativo sin necesidad de modificar el código.

- Se diseñó un ChallengeLoader, un gestor con patrón Singleton, responsable de persistir la selección del desafío del usuario entre escenas, asegurando que el contexto elegido en el menú se transfiera correctamente a la escena de programación.

■ **Implementación de la interfaz de usuario del menú:**

- Se diseñó e implementó la escena de menú principal (MenuScene), que incluye un panel de bienvenida y una sección para la selección de desafíos.
- Se crearon las tarjetas de desafío (ChallengeCardUI) como prefabs que se instancian dinámicamente para cada ChallengeData disponible, mostrando su título y descripción.
- Se integró la funcionalidad para que al hacer clic en una tarjeta, se pueda iniciar la carga del desafío correspondiente.

■ **Integración del contexto del desafío en la escena de programación:**

- Se adaptó el GameManager en la escena de programación (CodingScene) para que, al iniciarse, consultara al ChallengeLoader y obtuviera los datos del desafío seleccionado.
- La 'IDE_UIManager' se modificó para mostrar dinámicamente el título y las instrucciones del desafío activo, proporcionando al usuario un objetivo claro.
- Se añadió un cronómetro visual a la interfaz de simulación (ExecutionTimerView), que se activa durante la ejecución del programa, añadiendo un elemento de feedback y métrica sobre el rendimiento del programa del usuario.

■ **Creación de contenido inicial:**

- Se crearon los primeros 1-2 desafíos simples como activos ChallengeData, validando así todo el flujo del sistema, desde su selección en el menú hasta su resolución en el entorno de programación y simulación.
- **Hito Alcanzado:** El prototipo evolucionó de ser un simple editor de código a convertirse en un laboratorio virtual con un flujo educativo completo y estructurado. Se implementó un sistema robusto y escalable para la gestión de desafíos. El usuario ahora puede seleccionar una tarea, entender sus objetivos y tratar de resolverla en un entorno interactivo y con propósito, lo que representa la base del Producto Mínimo Viable (MVP) final.

Sprint 9: Simulador de código y robot

Duración: Del 19 de junio al 26 de junio de 2025.

Objetivo Principal: Realizar un pivote estratégico en el enfoque del proyecto ante los desafíos técnicos encontrados con la generación de bloques complejos. A propuesta del tutor, el nuevo objetivo era desarrollar un prototipo funcional basado en un sistema de programación híbrido, que combinara una estructura de bloques visual para la organización del programa con un lenguaje de comandos textual (DSL) para las acciones del robot.

Contexto y motivación (El desafío de los bloques tipo C): El trabajo realizado en sprints anteriores había permitido establecer una arquitectura MVC robusta y un sistema funcional de arrastre y conexión para bloques simples. Sin embargo, al intentar implementar bloques de control de flujo complejos (los denominados bloques de tipo C, como `if/else` o bucles), se encontraron obstáculos técnicos que excedían las estimaciones iniciales. La replicación de estos bloques requería un motor de layout dinámico extremadamente complejo, capaz de redimensionar y reorganizar geométricamente los bloques contenedores en tiempo real, una tarea de una magnitud considerable

Este desafío representó un punto de inflexión crítico, no solo desde una perspectiva técnica, sino también metodológica. Puso de manifiesto una realidad fundamental en cualquier proyecto de ingeniería: la necesidad de tomar decisiones importantes para asegurar la entrega de un Producto Mínimo Viable (MVP) funcional cuando surgen complejidades imprevistas.

El considerable esfuerzo y las horas dedicadas a la investigación y al desarrollo de la arquitectura inicial no podían quedar en un proyecto inacabado.

Asimismo, este momento subrayó la enorme dificultad que implica el desarrollo de un sistema complejo por una única persona. Fue la tenacidad y la convicción en el valor del trabajo realizado hasta la fecha lo que impulsó la búsqueda de una solución alternativa. En lugar de percibir el obstáculo como un fracaso, se abordó como una oportunidad para redefinir el problema y pivotar estratégicamente.

La nueva dirección, por tanto, no fue un abandono de los objetivos, sino una decisión consciente y pragmática. Se optó por un sistema híbrido que, manteniendo la claridad visual de un programa estructurado, introdujera la codificación textual manteniendo la programación por bloques adaptando el diseño de bloques al código diseñado en el repositorio de referencia. Este reenfoque permitió aprovechar todo el trabajo de base realizado (simulador 3D, arquitectura de control del robot) y centrar el esfuerzo restante en un nuevo núcleo técnico, el diseño de un DSL y su intérprete, que aportaría un valor pedagógico y técnico igualmente significativo.

Tareas clave realizadas y resultados:

- **Diseño e implementación del intérprete de código textual:**

- **Definición del DSL:** Se diseñó la gramática para el lenguaje de comandos del robot, incluyendo mover, girar, y, crucialmente, las estructuras de control repetir N veces, repetir por_siempre y su correspondiente cierre *fin_repetir*.
- **Creación del parser (**CommandParser**):** Se implementó el analizador que procesa el código fuente textual. Este componente valida la sintaxis, identifica los comandos y sus argumentos, y gestiona las estructuras anidadas mediante una pila (Stack), traduciendo el código a una lista de objetos Instruction.

- **Integración del nuevo flujo de ejecución:**

- Se modificó el **GameManager** para orquestar el nuevo flujo: UI -> Parser -> Ejecutor.

- El `RobotController` se adaptó para recibir y procesar la `'List<Instruction>'` generada por el parser, implementando una corrutina (`'ProcessInstructionList'`) capaz de ejecutar tanto comandos simples como bucles anidados de forma recursiva.

■ **Implementación del sistema de desafíos y escenas:**

- Se creó el sistema de `ChallengeData` (Scriptable Objects) y `ChallengeLoader` para gestionar la selección de desafíos desde una nueva escena de menú (`MenuScene`).
- Se desarrollaron las primeras versiones de la **escena de introducción** y el menú de selección, estableciendo el flujo completo de la aplicación, desde el inicio hasta la programación y simulación.

■ **Pruebas del sistema híbrido:**

- Se realizaron pruebas exhaustivas del nuevo sistema, validando que el robot ejecutaba correctamente secuencias complejas de comandos textuales, incluyendo bucles finitos e infinitos.
- Se implementó la lógica de detección de colisiones (`CollisionTextLogger`, `MuroController`) para que el robot se detuviera al chocar, añadiendo una capa de interacción realista con el entorno.

■ **Diseño de una interfaz de programación híbrida (`IDE_UIManager`):**

Se rediseñó la interfaz de usuario para dar cabida a dos modos de programación interconectados, sirviendo como un puente entre la codificación visual y la textual.

- **Modo de programación textual (basado en DSL):** Se implementó una nueva vista para que los usuarios pudieran aprender a programar con código escrito y ver su ejecución inmediata en la simulación del robot. Esta vista incluye:
 - Un campo de entrada de texto donde el usuario introduce los comandos del DSL.
 - Un sistema que, al añadir un comando, instancia un prefab visual (`commandBlockPrefab`), creando una representación secuencial del programa que mantiene la claridad de un sistema de bloques.

- La reconfiguración del botón ejecutar para enviar el código fuente completo al `CommandParser` para su análisis y posterior ejecución.
- **Modo de programación visual (Basado en UBlockly):** En lugar de descartar el trabajo previo, este se adaptó y se integró como una segunda modalidad de programación. Basándose en la arquitectura MVC y desarrollando nuevos controladores y eliminando los anteriores y se configuró un espacio de trabajo que permite:
 - Arrastrar y conectar los bloques de evento y movimiento (`motion_movesteps`) que ya eran funcionales.
 - Ejecutar las secuencias de bloques directamente a través del motor de UBlockly (`CSharp.Runner`), traduciendo las acciones al mismo `RobotController` utilizado por el modo textual.
 - uso de bloques tipo C para repetir X veces.

Esta doble funcionalidad permite al usuario elegir su método de aprendizaje preferido o incluso transicionar de uno a otro, validando la flexibilidad de la arquitectura y la viabilidad de ambos enfoques dentro del mismo prototipo.

Hito alcanzado: Se completó con éxito el pivote estratégico del proyecto. El resultado es un prototipo MVP completamente funcional basado en un sistema híbrido y original, con un intérprete de comandos propio y un flujo de desafíos estructurado. Este hito no solo representa una solución técnica robusta a los problemas encontrados, sino también un enfoque pedagógico innovador que sirve de puente entre la programación visual y la textual.

Sprint 10: Revisión producto mínimo viable y memoria TFG

Duración: Del 26 de junio al 4 de julio de 2025.

Objetivo principal: Realizar una revisión exhaustiva del prototipo para asegurar su estabilidad y pulir la experiencia de usuario. Completar la redacción final de la memoria y sus anexos, garantizando la coherencia, precisión y calidad del documento para su entrega y defensa.

Contexto y motivación: Con el producto mínimo viable (MVP) híbrido completamente funcional, este último sprint se dedicó a la fase de estabilización y empaquetado. El enfoque se desplazó del desarrollo de nuevas funcionalidades a la mejora de la calidad del producto existente y a la producción de una documentación técnica que reflejara con exactitud el trabajo realizado, los desafíos superados y los aprendizajes obtenidos.

Tareas clave realizadas y resultados:

■ **Refinamiento y limpieza del código fuente:**

- Se llevó a cabo una revisión integral del repositorio en GitHub, eliminando código obsoleto o comentado que ya no era relevante para la versión final del proyecto.
- Se optimizaron y refactorizaron pequeñas partes del código para mejorar su legibilidad y eficiencia, asegurando que la base de código fuera limpia y comprensible.
- Se verificó que todos los comentarios en el código fueran pertinentes y aportaran valor para entender la lógica implementada.

■ **Pulido de la interfaz de usuario y la experiencia (UX):**

- Se realizó una revisión completa de todos los textos de la interfaz de usuario (IDE_UIManager, menús, descripciones de desafíos), corrigiendo erratas y mejorando la claridad de los mensajes para el usuario final.
- Se ajustó la disposición y el anclaje de elementos visuales, como imágenes y botones, para asegurar una apariencia consistente y profesional en diferentes resoluciones de pantalla.

■ **Consolidación de la memoria del TFG:**

- Se redactó la versión final de todos los capítulos de la memoria, asegurando que la narrativa del pivote estratégico y el sistema híbrido fuera coherente a lo largo de todo el documento con respecto a la versión que se realizó en otro sprint anterior.
- Se revisaron y corrigieron todas las secciones para mejorar la cohesión, la claridad y la precisión técnica, incorporando el feedback del tutor.

- Se regeneraron todas las figuras (diagramas, capturas de pantalla) y tablas, asegurando que estuvieran correctamente numeradas, etiquetadas y referenciadas en el texto.
- Se revisó la bibliografía, verificando que todas las citas estuvieran correctamente incluidas.

■ **Completitud de los anexos:**

- Se finalizó la redacción de los diferentes anexos que se incluyen en el Trabajo Final de Grado.

Hito alcanzado: Entrega de un MVP estable, pulido y funcional. Se completó la versión final de la memoria del TFG y sus anexos, listos para su presentación. El proyecto se encuentra en un estado consolidado que no solo cumple con los objetivos redefinidos, sino que también documenta de manera transparente y profesional todo el ciclo de vida del desarrollo.

A.3. Estudio de viabilidad

Viabilidad económica

Dado que este proyecto se desarrolla en el marco de un Trabajo Fin de Grado, los costes económicos se limitan principalmente a los recursos de hardware y software ya disponibles por el estudiante y la universidad, y a servicios de bajo o nulo coste para desarrolladores individuales.

Costes de personal

El desarrollo ha sido realizado por un único estudiante como parte de su formación académica. No se contemplan costes salariales directos asociados al desarrollo del prototipo. El tiempo dedicado se considera inversión en aprendizaje y cumplimiento de los requisitos del TFG. La labor de tutorización por parte del profesor D. Carlos López Nozal se enmarca dentro de sus funciones docentes en la Universidad de Burgos.

Costes de hardware

- **Equipo de desarrollo:** Se ha utilizado un ordenador personal estándar con capacidad suficiente para ejecutar Unity, Blender y las herramientas de desarrollo.

- **Dispositivo de prueba (Tablet Android):** Para las pruebas de portabilidad y usabilidad, se ha utilizado una tablet Android genérica.
- **Coste estimado de amortización de hardware (simulado):** Si se tuviera que estimar un coste de amortización para un proyecto independiente, se podría considerar un valor simbólico, pero para este TFG, los costes directos son nulos.

Costes de software y servicios

- **Unity 6 (Personal Edition):** Licencia gratuita para estudiantes y desarrolladores individuales.
- **C# y Visual Studio Community:** Herramientas gratuitas.
- **UBlockly:** Repositorio de código abierto, sin coste de licencia.
- **Blender:** Software de modelado 3D de código abierto y gratuito.
- **LEGO Digital Designer / Studio 2.0:** Software gratuito de LEGO para diseño.
- **GitHub:** Plan gratuito para repositorios públicos o privados (si aplica cuenta de estudiante).
- **Zube:** Ofrece planes gratuitos o de bajo coste para pequeños equipos o uso individual en proyectos de código abierto. Se ha utilizado bajo estas condiciones o como herramienta de prueba.

En resumen, los costes económicos directos para el desarrollo del prototipo han sido **mínimos o nulos**, aprovechando licencias educativas, software de código abierto y recursos preexistentes.

Viabilidad legal y ética

Licencias de software

- **Unity Personal Edition:** Su licencia permite el desarrollo y distribución de proyectos siempre que no se superen ciertos umbrales de ingresos o financiación, lo cual no aplica a este TFG.

- **UBlockly:** Generalmente distribuido bajo una licencia permisiva de código abierto (Apache 2.0). Se ha respetado la licencia original al adaptar el código, manteniendo las atribuciones correspondientes si fueran necesarias (aunque en este caso, la adaptación es para un TFG y no para redistribución comercial del UBlockly original).
- **Otros componentes (Blender, etc.):** Se han utilizado bajo sus respectivas licencias de código abierto.

Propiedad intelectual

El código fuente desarrollado específicamente para este TFG, así como los diseños y modelos 3D originales creados, son propiedad intelectual del autor (Luis Ignacio de Luna Gómez) y de la Universidad de Burgos, según la normativa aplicable a Trabajos Fin de Grado. La adaptación del código de UBlockly se ha realizado con el fin de crear una nueva obra derivada para un propósito académico y de investigación.

Privacidad de datos

Los requisitos de normativas de protección de datos como GDPR son mínimos en esta fase. Si en futuras líneas de trabajo se integrasen servicios en la nube que manejen datos de usuario, sería necesario realizar un análisis exhaustivo y aplicar las políticas de privacidad correspondientes.

Consideraciones éticas

El proyecto tiene un fin educativo y busca promover el acceso a la tecnología. Se ha evitado el uso de assets o recursos con derechos de autor restrictivos sin la debida licencia. El diseño de la interfaz y la simulación buscan ser inclusivos y no discriminatorios.

Viabilidad técnica

La viabilidad técnica del prototipo se ha demostrado mediante su implementación funcional.

- **Tecnologías maduras:** Unity y C# son tecnologías robustas y ampliamente documentadas, lo que ha facilitado el desarrollo.

- **Adaptabilidad de UBlockly:** A pesar de no estar diseñado para replicar Scratch directamente, la arquitectura de UBlockly ha demostrado ser lo suficientemente flexible para ser adaptada a los requisitos del proyecto mediante la implementación de un patrón MVC y la personalización de vistas y controladores.
- **Limitaciones del prototipo:** El prototipo actual implementa un conjunto limitado de bloques y funcionalidades. La viabilidad de expandirlo a un sistema completo con todas las características de Scratch dependería de un esfuerzo de desarrollo continuado, pero las bases técnicas establecidas son sólidas.
- **Rendimiento:** El prototipo, con su alcance actual, funciona de manera fluida en un equipo de desarrollo estándar y se espera un rendimiento aceptable en tablets Android de gama media, aunque esto requeriría pruebas específicas en la plataforma objetivo como parte de una línea futura.

En conclusión, el proyecto se considera viable tanto económica como legal y técnicamente dentro del contexto de un Trabajo Fin de Grado enfocado en el desarrollo de un prototipo funcional.

Apéndice B

Especificación de Requisitos

B.1. Introducción

Dentro de este apéndice, se detalla la especificación de requisitos del proyecto del laboratorio virtual de programación y robótica. Una adecuada definición y gestión de los requisitos es fundamental en ingeniería de software para asegurar que el sistema desarrollado satisfaga las necesidades y expectativas de los usuarios, así como para guiar las fases posteriores de diseño e implementación. Dada la naturaleza de prototipo del presente Trabajo Fin de Grado, esta especificación se centrará en los requisitos que guían la funcionalidad actual del prototipo, así como aquellos objetivos clave que sentarán las bases para futuras expansiones. Los requisitos se clasifican en funcionales (qué debe hacer el sistema) y no funcionales (cómo debe funcionar, relativo a la calidad).

B.2. Objetivos generales del proyecto

Los objetivos generales que guían el desarrollo de este laboratorio virtual han sido definidos previamente en el Capítulo 2 de la memoria. Estos objetivos, que se persiguen tanto a nivel funcional como no funcional, se sintetizan a continuación:

- **Desarrollar un prototipo de laboratorio virtual de programación y robótica** que permita a los estudiantes aprender programación mediante bloques y código escrito.

- **Replicar la experiencia de usuario y la lógica fundamental de Scratch** en un entorno Unity para después ejecutar una aplicación en un dispositivo android.
- **Integrar el sistema de bloques con una simulación 3D en Unity** para la visualización de acciones del robot.
- **Implementar una simulación básica de un robot LEGO WeDo 2.0** en el entorno virtual.
- **Garantizar la accesibilidad y portabilidad del prototipo** a plataformas Android (Tablets).
- **Investigar y analizar plataformas existentes** para informar futuras mejoras.
- **Adquirir conocimientos técnicos** en Unity y C# en el contexto del desarrollo de programación visual y simulación.

B.3. Catálogo de requisitos

El siguiente catálogo enumera los requisitos clave del prototipo. Se organizan por tipo y se proporciona un identificador único (RF para Requisito Funcional, RNF para Requisito No Funcional) y una descripción concisa del mismo.

Requisitos funcionales (RF)

Estos requisitos describen las funcionalidades específicas que el laboratorio virtual debe proporcionar al usuario.

Gestión del entorno y desafíos

- RF-001 Gestión de la interfaz principal:** El sistema debe presentar una interfaz de usuario que distinga claramente entre el área de selección de herramientas (desafíos) y el área principal de codificación (workspace).
- RF-002 Selección de desafíos:** El usuario debe poder seleccionar un desafío desde un menú, y el sistema debe cargar el contexto y las instrucciones correspondientes en la interfaz de programación.

- RF-003 Gestión de categorías de bloques:** El usuario debe poder seleccionar una categoría para visualizar un conjunto filtrado de bloques de programación en una toolbox.
- RF-004 Arrastre de bloques:** El usuario debe poder seleccionar un bloque de la toolbox y arrastrarlo hasta el área de codificación.
- RF-005 Conexión y apilamiento de bloques:** El sistema debe permitir la unión visual (snap) y lógica de bloques compatibles, representando correctamente la jerarquía secuencial de un programa.
- RF-006 Descarte de bloques:** El usuario debe poder eliminar bloques del espacio de trabajo arrastrándolos fuera del área de codificación.
- RF-007 Entrada de comandos textuales:** El sistema debe proporcionar una interfaz (un campo de texto) para que el usuario pueda escribir comandos según la sintaxis de un lenguaje de dominio específico (DSL).
- RF-008 Visualización secuencial del código:** Cada comando textual añadido debe ser representado visualmente como un ítem en una lista secuencial, manteniendo una estructura clara y legible del programa.
- RF-009 Parseo y validación del DSL:** El sistema debe analizar (parsear) el código textual, validar su sintaxis y proporcionar feedback inmediato y claro al usuario en caso de error.
- RF-010 Limpieza del programa textual:** El usuario debe disponer de una opción para borrar todos los comandos textuales escritos y empezar de nuevo.
- RF-011 Ejecución unificada:** El usuario debe poder iniciar la ejecución del programa completo (sea por bloques, textual o híbrido) a través de una única acción.
- RF-012 Interpretación y simulación del robot:** El motor de ejecución debe interpretar la secuencia de instrucciones (generada a partir de los bloques o del DSL) y traducirla en acciones visibles y coherentes del robot en el entorno 3D.
- RF-013 Detección de colisiones:** El sistema debe detectar colisiones del robot con elementos del entorno y detener la ejecución del programa.
- RF-014 Detención manual de la ejecución:** El usuario debe poder detener la simulación en cualquier momento a través de un control en la interfaz.

Requisitos no funcionales (RNF)

Estos requisitos describen las cualidades del sistema que garantizan un buen funcionamiento y una experiencia de usuario óptima.

1. **RNF-001: Rendimiento de la interfaz.** La interfaz de programación visual debe responder a las interacciones del usuario (arrastre, conexión) con una latencia mínima, garantizando una experiencia fluida.
2. **RNF-002: Consistencia visual.** El diseño visual de los bloques y la interfaz debe ser consistente con la estética de Scratch.
3. **RNF-003: Accesibilidad.** El sistema debe ser capaz de ejecutarse en dispositivos Android tipo tablet con hardware común, sin requerimientos técnicos elevados.
4. **RNF-004: Precisión de la simulación.** El movimiento y el comportamiento del robot simulado deben ser perceptibles y acordes con la lógica programada en los bloques.
5. **RNF-005: Mantenibilidad.** La arquitectura del software debe facilitar la futura modificación y extensión del sistema de bloques y la lógica del robot.
6. **RNF-006: Confiabilidad.** El sistema no debe experimentar caídas o bloqueos inesperados durante la interacción normal o la ejecución de programas.
7. **RNF-007: Modularidad del código.** La arquitectura del software del sistema debe estar organizada siguiendo el patrón Modelo-Vista-Controlador (MVC), promoviendo una clara separación de responsabilidades. Los componentes dentro de cada capa (Modelo, Vista y Controlador) deben diseñarse con principios de *bajo acoplamiento* (interdependencia mínima entre módulos) y *alta cohesión* (elementos dentro de un módulo con funcionalidades estrechamente relacionadas). Esta estructura facilitará la comprensión del código, la depuración, la

modificación aislada de componentes y el mantenimiento general del sistema.

8. **RNF-008: Extensibilidad para nuevos bloques.** El diseño del sistema debe permitir la adición de nuevos tipos de bloques de programación con un impacto mínimo y localizado en el código fuente existente. La incorporación de un nuevo bloque debería, idealmente, implicar la creación de:

- Una definición de bloque.
- Una clase para su modelo lógico .
- Un prefab para su representación visual en Unity UI .
- Una clase intérprete si el bloque es ejecutable.

Estos nuevos componentes deben poder integrarse sin requerir modificaciones extensas en el núcleo del sistema o los controladores principales.

9. **RNF-009: Usabilidad y retroalimentación visual clara.** El sistema debe proporcionar indicaciones visuales claras e intuitivas al usuario durante las interacciones clave para facilitar su uso y comprensión:

- **Resaltado de conexiones válidas:** Durante la operación de arrastre de un bloque, las posibles conexiones compatibles con otros bloques deben resaltarse visualmente (ej. mediante un cambio de color o una superposición gráfica en el punto de snap) para guiar al usuario hacia una unión correcta.
- **Retroalimentación ante conexiones inválidas:** Si el usuario intenta realizar una conexión entre bloques incompatibles, el sistema debería, en la medida de lo posible, indicar visualmente que la conexión no es permitida.
- **Indicación de ejecución:** El bloque que se está ejecutando actualmente debe ser visualmente distinguible para que el usuario pueda seguir el flujo del programa.

B.4. Especificación de requisitos

Actores del sistema

Un actor representa un rol desempeñado por un usuario humano, un dispositivo hardware o incluso otro sistema software que interactúa con el sistema. Para el presente prototipo, se identifican los siguientes actores principales:

Actor A01: Estudiante (Usuario principal)	
Descripción:	Individuo (generalmente un niño, adolescente o persona en formación) que utiliza el laboratorio virtual para aprender los fundamentos de la programación y la robótica mediante la creación y ejecución de programas. La plataforma le permite interactuar tanto con bloques visuales para la estructura del programa como con un lenguaje de comandos textual para definir acciones específicas.
Tipo:	Humano.
Objetivo Principal:	Resolver desafíos de robótica mediante la construcción de programas híbridos para controlar un robot virtual, observando su comportamiento en un entorno de simulación 3D. El fin último es desarrollar habilidades de pensamiento computacional y facilitar la transición de la programación visual a la textual.
Relaciones con Casos de Uso Principales:	■ Seleccionar un desafío de aprendizaje. ■ Construir programas de forma híbrida (bloques y texto). ■ Ejecutar y simular el comportamiento del robot.

Tabla B.1: Descripción del Actor A01 - Estudiante.

Casos de uso

A continuación, se detallan los casos de uso (CU) que describen las interacciones clave entre los actores y el sistema. Cada caso de uso se presentará en una tabla siguiendo un formato estándar.

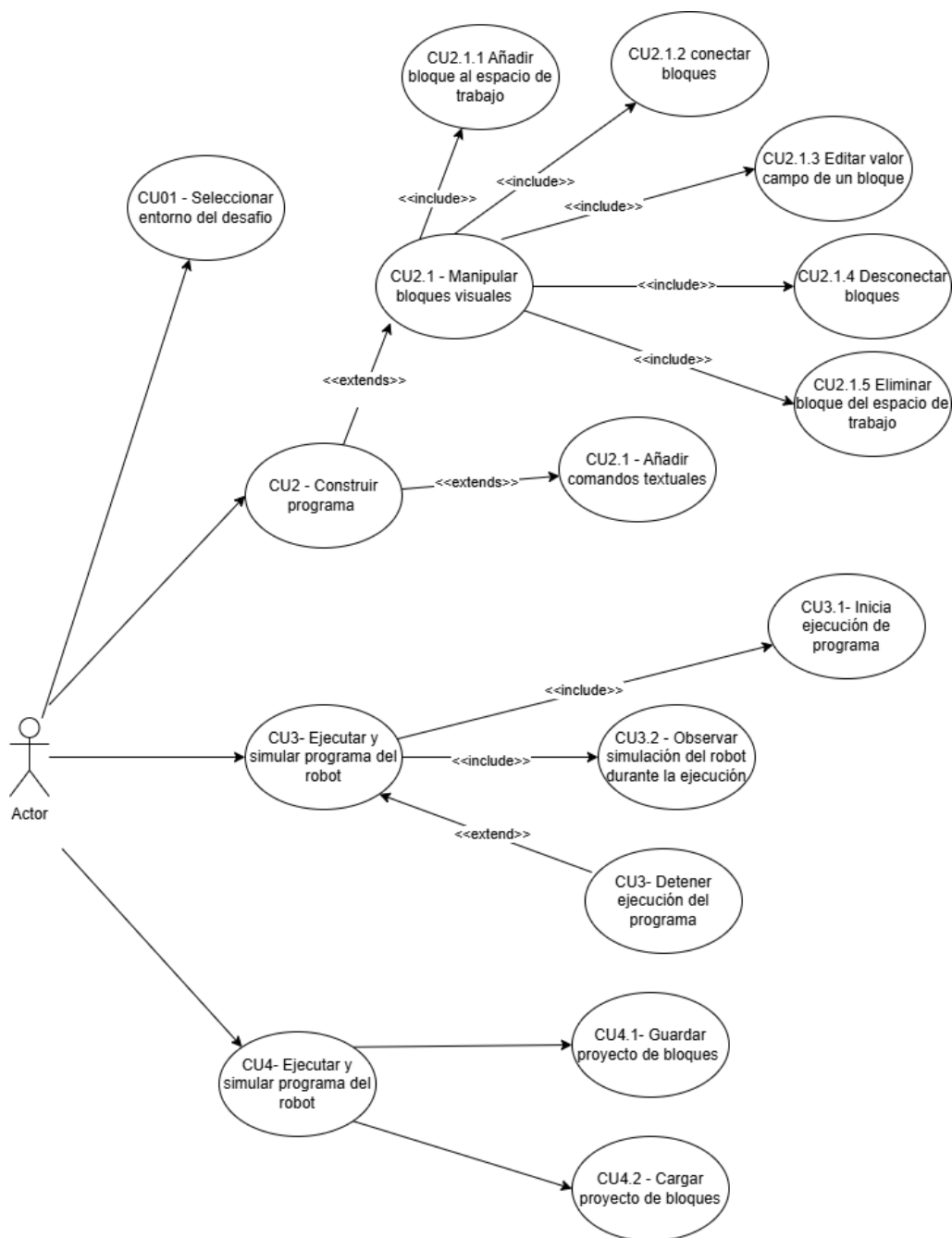


Figura B.1: Diagrama de caso de uso CU - Diagrama caso de uso de la aplicación.

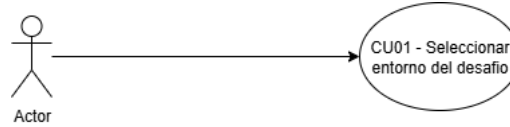
CU-01: Seleccionar entorno del desafío

Figura B.2: Diagrama de caso de uso CU - Diagrama seleccionar entorno del desafío.

Versión:	1.0
Actor:	A01 - Estudiante
RF asociados:	RF-001, RF-002
Descripción:	El estudiante navega por el menú principal para seleccionar un desafío específico, lo que prepara el entorno de programación con los objetivos y la configuración necesarios.
Precondición:	El estudiante se encuentra en la escena del menú principal de la aplicación.
Acciones:	1. El sistema muestra al estudiante una lista de desafíos disponibles. 2. El estudiante selecciona uno de los desafíos. 3. El sistema carga la información del desafío seleccionado (título, descripción). 4. El sistema transiciona a la escena de programación, pasando el contexto del desafío.
Postcondición:	La interfaz de programación se muestra con las instrucciones y el entorno preparados para el desafío seleccionado.
Importancia:	Alta

Tabla B.2: CU-01: Seleccionar Desafío de Aprendizaje.

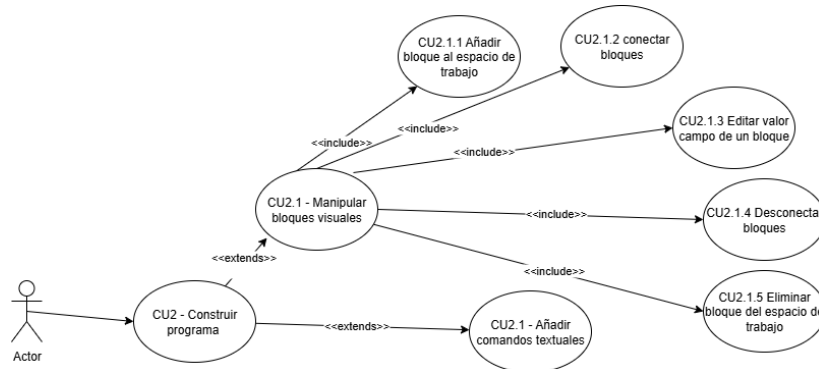
CU-02: Construir programa

Figura B.3: Diagrama de caso de uso CU - Diagrama construir programa.

Versión:	2.0 (Híbrida)
Actor:	A01 - Estudiante
RF asociados:	RF-003 a RF-010, RF-014 (para texto), RF-017 (para bloques)
Descripción:	El estudiante utiliza las herramientas de la interfaz para construir un programa que controle al robot. Puede hacerlo con bloques visuales, o con comandos textuales.
Precondición:	El estudiante se encuentra en la escena de programación.
Flujo Principal:	El estudiante realiza una o más de las siguientes acciones de forma iterativa: ■ Añadir y manipular bloques visuales (Extiende a CU-02.1). ■ Escribir y añadir comandos textuales (Extiende a CU-02.2).
Postcondición:	Se ha construido o modificado un programa en el área de trabajo, listo para ser ejecutado.
Importancia:	Alta

Tabla B.3: CU-02: Construir programa.

CU-02.1: Manipular bloques visuales

Versión:	1.1
Extiende a:	CU-02: Construir programa
Descripción:	El estudiante utiliza el paradigma de programación visual para seleccionar, arrastrar, conectar y configurar bloques, formando la estructura y lógica del programa.
Acciones:	El Estudiante puede realizar cualquiera de las siguientes acciones: ■ Añadir un nuevo bloque: Selecciona un bloque de la toolbox y lo arrastra al área de codificación. ■ Conectar bloques: Une bloques compatibles mediante drag-and-drop, observando el feedback visual (resaltado y snap). ■ Editar campos: Modifica valores en los campos de un bloque (ej. número de pasos). ■ Desconectar y Reorganizar bloques: Separa bloques de una pila para reordenar la lógica. ■ Eliminar bloques: Descarta bloques no deseados arrastrándolos fuera del área de trabajo.

Tabla B.4: CU-02.1: Manipular bloques visuales.

CU-02.2: Añadir comandos textuales

Versión:	1.0
Extiende a:	CU-02: Construir programa
RF asociados:	RF-007, RF-008, RF-009, RF-010
Descripción:	El estudiante utiliza la interfaz de programación textual para definir secuencias de acciones del robot.
Precondición:	El estudiante está en la vista de programación textual o utilizando un bloque que permite la inserción de código.
Acciones:	1. El estudiante escribe un comando del DSL (ej. mover 10 pasos) en el campo de texto de la IDE. 2. Pulsa el botón Añadir comando (+). 3. Si es válido, el sistema añade el comando al programa, creando una nueva representación visual en la lista de instrucciones.
Postcondición:	El programa ha sido modificado con la nueva instrucción textual.
Importancia:	Alta

Tabla B.5: CU-02.2: Añadir comandos textuales.

CU-03: Ejecutar y simular programa

Versión:	1.0
Actor:	A01 - Estudiante
RF asociados:	RF-006, RF-008, RF-009, RF-010, RF-012
Descripción:	El estudiante inicia la ejecución de su programa para validar su lógica y observar el comportamiento del robot en el entorno 3D.
Precondición:	Existe un programa (bloques, texto o híbrido) en el área de trabajo.
Acciones:	1. El estudiante pulsa el botón ejecutar. 2. El sistema analiza el programa. 3. Flujo alternativo (error): Si el código textual contiene un error de sintaxis, el sistema muestra un mensaje de error en la interfaz y la ejecución se detiene. 4. Si el programa es válido, el sistema inicia la simulación. 5. El motor de ejecución interpreta la secuencia de bloques y comandos, y el robot virtual realiza las acciones correspondientes en la escena 3D. 6. El estudiante observa el resultado. 7. La ejecución finaliza (por completitud, colisión o detención manual), y el sistema muestra un mensaje de resultado.
Postcondición:	El estudiante ha recibido feedback sobre la ejecución de su programa y puede volver al modo de edición.
Importancia:	Alta

Tabla B.6: CU-03: Ejecutar y Probar Programa.

CU03.1: Iniciar ejecución de programa

Versión:	1.0
Actor:	A01 - Estudiante
RF asociados:	RF-007, RF-009 (parcial), RF-011 (parcial)
Descripción:	El estudiante activa el mecanismo para comenzar la ejecución de su programa, provocando la transición al modo de simulación y la preparación del entorno.
Precondición:	■ El Estudiante ha construido un programa en el área de codificación que. ■ El sistema se encuentra en el modo de programación.
Acciones:	■ El Estudiante pulsa el botón/icono en la interfaz principal para que el programa se ejecute. ■ El sistema realiza la transición al modo de simulación: • En el caso de programación por bloques oculta los paneles de edición de bloques (categorías, toolbox, área de codificación). • Activa y muestra el entorno de simulación 3D. • El robot virtual se inicializa en la escena (posición/orientación por defecto). • Actualiza los controles de la interfaz y el botón stop se activa mientras el botón bandera verde se desactiva).
Postcondición:	■ El sistema está en modo de simulación, con el entorno 3D y el robot visibles. ■ La cuenta atrás ha finalizado (o está a punto de finalizar), preparando el inicio de la interpretación de bloques.
Importancia:	Alta

Tabla B.7: CU03.1 - Iniciar ejecución de programa.

CU03.2: Observar simulación del robot durante ejecución

Versión:	1.0
Actor:	A01 - Estudiante
RF asociados:	RF-009, RF-010, RF-011 (durante ejecución), RF-014
Descripción :	Una vez iniciada la ejecución, el estudiante observa cómo el sistema interpreta los bloques y cómo el robot virtual responde a dichas instrucciones en el entorno 3D.
Precondición:	■ La ejecución del programa ha sido iniciada correctamente . ■ El sistema está en modo de simulación y el programa está activo.
Acciones:	■ El sistema interpreta secuencialmente cada bloque del programa, comenzando desde el anclado al evento de bandera verde. ■ Para cada bloque interpretado: • El intérprete extrae el parámetro numérico de pasos. • El robot virtual ejecuta un movimiento lineal correspondiente en la escena 3D, con una animación suave. ■ El sistema puede mostrar mensajes textuales en la UI (Robot moviendo X pasos...). ■ El estudiante observa estas acciones y el feedback visual. ■ El flujo continúa hasta que se alcanza el final de la secuencia de bloques o hasta que el usuario interrumpe la ejecución.
Postcondición:	■ El Estudiante ha podido observar el comportamiento del robot en respuesta a su programa. ■ El programa ha finalizado su ejecución natural, o sigue en curso.
Importancia:	Alta

Tabla B.8: CU03.2 - Observar simulación del robot durante ejecución.

CU03.3: Detener ejecución de programa

Versión:	1.0
Actor:	A01 - Estudiante
RF asociados:	RF-008
Descripción:	El estudiante interrumpe la ejecución en curso de su programa de bloques y la simulación del robot.
Precondición:	■ El programa de bloques está actualmente en ejecución en el modo de simulación. ■ El botón/icono stop es visible y está activo.
Acciones:	■ El Estudiante pulsa el botón/icono stop en la interfaz. ■ El sistema detiene la interpretación de bloques. ■ Cualquier acción o animación del robot en curso se interrumpe. ■ El sistema realiza la transición de vuelta al modo de programación: • Restaura la visibilidad de los paneles de edición de bloques. • Oculta o desactiva el entorno de simulación 3D. • Actualiza los controles de la interfaz, el botón bandera verde se activa, el botón stop se desactiva).
Postcondición:	■ La ejecución del programa y la simulación del robot han sido detenidas. ■ El sistema ha retornado al modo de programación, permitiendo al Estudiante modificar su programa.
Importancia:	Alta

Tabla B.9: CU03.3 - Detener ejecución de programa.

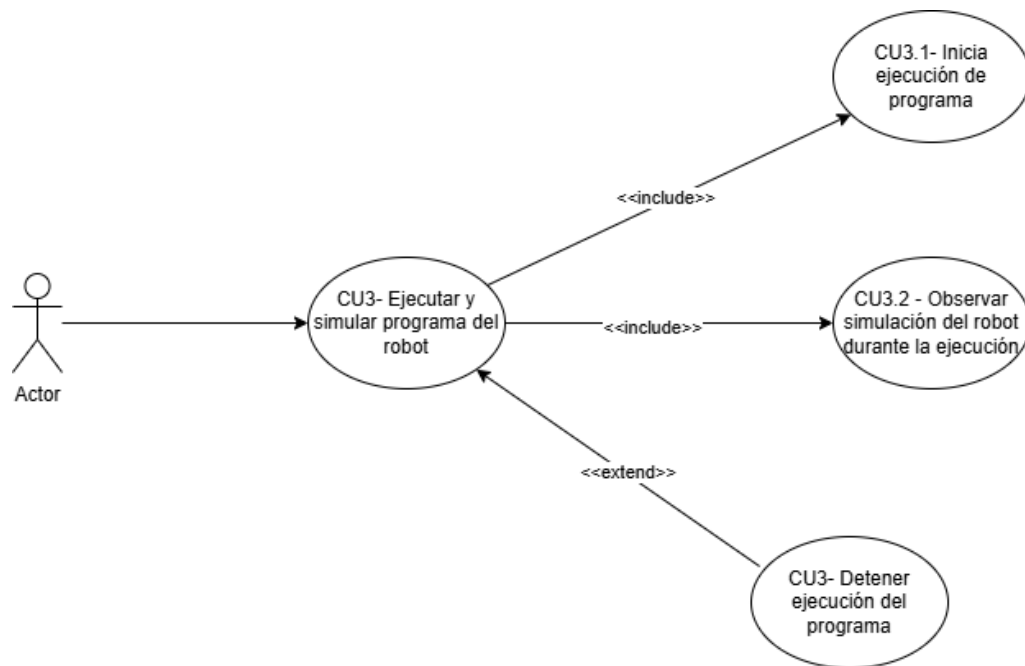


Figura B.4: Diagrama de caso de uso CU03- Ejecutar y simular programa del robot.

CU04: Gestionar proyectos de bloques

Versión:	1.0
Autor:	Luis Ignacio de Luna Gómez
Actor:	A01 - Estudiante
RF Asociados:	RF-012, RF-013, RF-015
Descripción :	El estudiante puede guardar su progreso en un programa de bloques y recuperar trabajos previos, permitiendo la continuidad entre sesiones de uso del laboratorio virtual.
Precondiciones :	■ El sistema ha sido iniciado correctamente y la interfaz principal es visible.
Acciones:	El Estudiante realiza una de las siguientes acciones: ■ Guardar el proyecto actual. ■ Cargar un proyecto existent. ■ El sistema almacena el XML generado de forma persistente.
Postcondición:	■ El estado del proyecto ha sido gestionado (guardado o cargado) según la acción del Estudiante.
Importancia:	Media

Tabla B.10: CU04 - Gestionar proyectos de bloques.

CU04.1: Guardar proyecto de bloques

Versión:	1.0
Actor(es) Principal(es):	A01 - Estudiante
RFs asociados:	RF-012
Descripción:	El estudiante guarda el estado actual de su programa de bloques para poder recuperarlo en una sesión posterior.
Precondiciones:	■ El Estudiante tiene un programa (nuevo o modificado) en el área de codificación. ■ La interfaz de programación está activa.
Acciones:	1. El Estudiante pulsa el botón/icono guardar en la interfaz principal. 2. El sistema serializa el estado actual del entorno de trabajo (incluyendo todos los bloques, sus conexiones y los valores de sus campos) a un formato XML. 3. El sistema almacena el XML generado de forma persistente en el almacenamiento local del dispositivo. 4. (Opcional) El sistema muestra un mensaje de confirmación al Estudiante
Postcondiciones:	■ El estado actual del programa de bloques del Estudiante está guardado persistentemente en el dispositivo. ■ El Estudiante puede continuar trabajando o cerrar la aplicación con la seguridad de que su progreso está almacenado.
Importancia:	Media

Tabla B.11: CU04.1 - Guardar proyecto de bloques.

CU04.2: Cargar proyecto de bloques

Versión:	1.0
Actor:	A01 - Estudiante
RF asociados:	RF-012, RF-013 y RF-015
Descripción:	El estudiante carga un programa de bloques previamente guardado para continuar trabajando en él o ejecutarlo.
Precondición:	■ Existe al menos un proyecto previamente guardado por el sistema en el almacenamiento local. ■ La interfaz de programación está activa.
Acciones:	■ El Estudiante pulsa el botón/icono cargar en la interfaz principal. ■ El sistema recupera los datos XML del proyecto guardado desde el almacenamiento local. ■ El sistema limpia el espacio de trabajo actual (eliminando todos los bloques existentes) y las vistas correspondientes del área de codificación. ■ El sistema deserializa el contenido XML y reconstruye la estructura lógica del entorno de trabajo, incluyendo todos los bloques, sus conexiones y los valores de sus campos. ■ El sistema genera y muestra las vistas correspondientes en el área de codificación, reflejando el estado del programa cargado.
Postcondición:	■ El área de codificación muestra el programa de bloques que fue previamente guardado. ■ El Estudiante puede continuar editando o ejecutar el programa cargado.
Importancia:	Media

Tabla B.12: CU04.2 - Cargar proyecto de bloques.

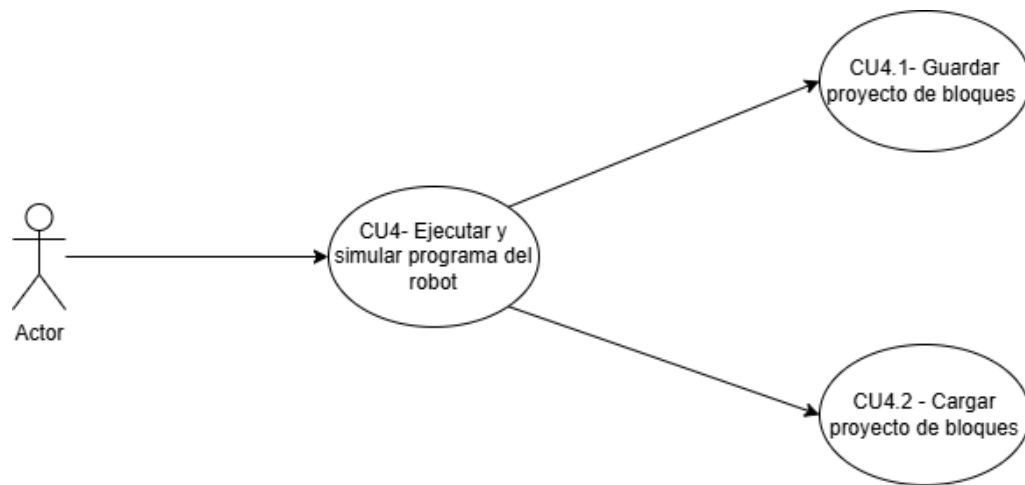


Figura B.5: Diagrama de caso de uso CU04 - Gestionar proyectos de bloques.

Apéndice C

Especificación de Diseño

C.1. Introducción

Este apéndice presenta la especificación de diseño del prototipo del laboratorio virtual de programación y robótica. El diseño del software es una fase crítica que traduce los requisitos funcionales y no funcionales, detallados en el Apéndice B, en una descripción técnica y estructurada de la solución. El propósito de este documento es detallar la arquitectura del sistema, el diseño de los datos que maneja, y el diseño procedimental de sus interacciones clave.

Se describirá tanto el **diseño inicial basado en un sistema de bloques visuales**, que sentó las bases conceptuales y arquitectónicas del proyecto, como el **diseño final del prototipo híbrido**, que representa la solución implementada.

Se busca proporcionar una visión clara de cómo se han organizado los componentes del software, cómo interactúan entre sí y cómo se ha estructurado la lógica de la aplicación para cumplir con los objetivos del proyecto. Este diseño se ha enfocado en la modularidad, mantenibilidad y escalabilidad, sentando las bases para futuras expansiones del laboratorio virtual. Se presta especial atención a la adaptación de la librería UBlockly dentro del motor Unity y la implementación del patrón Modelo-Vista-Controlador (MVC).

C.2. Diseño del sistema de programación inicial (basado en bloques)

Diseño de datos del modelo de bloques

El diseño de datos describe las principales entidades de información que el sistema gestiona, sus atributos y las relaciones entre ellas. Este diseño es fundamental para la persistencia, la lógica de negocio y la representación visual de los elementos del programa.

Entidades principales del modelo

El sistema se articula en torno a un conjunto de entidades modelo que representan los conceptos clave de la programación por bloques y la gestión del espacio de trabajo, heredadas y adaptadas de la estructura de UBlockly:

- **Workspace:** Representa el contenedor principal de todo el entorno de programación.
 - **Atributos Clave:** `Id` (identificador), `Options` (configuración), `TopBlocks` (lista de bloques raíz), `BlockDB` (diccionario de todos los bloques), `VariableMap` (gestor de variables), `ProcedureDB` (gestor de procedimientos), `ConnectionDBList` (bases de datos de conexiones optimizadas para búsqueda).
 - **Relaciones:** Contiene múltiples `BlockModels` y gestiona las colecciones globales de variables y procedimientos.
- **Block:** Representa la estructura lógica y el estado de un bloque individual de programación.
 - **Atributos Clave:** `ID`, `Type` (ej. `motion_movesteps`), `XY` (posición lógica), `InputList`, `OutputConnection`, `PreviousConnection`, `NextConnection`, propiedades de estado (`Disabled`, `Movable`, `Collapsed`, etc.).
 - **Relaciones:** Puede tener un `ParentBlock` y múltiples `ChildBlocks` (a través de `NextConnection` o `InputModel` de tipo sentencia). Se conecta a otros `BlockModels` mediante sus `ConnectionModels`.
- **Connection:** Define un punto de conexión en un bloque.

- **Atributos Clave:** `SourceBlock`, `Type` (`EConnection`), `TargetConnection`, `Check` (tipos compatibles), `Location` (posición lógica para snapping), `InDB`, `DB`, `DBOpposite`.
- **Relaciones:** Pertenece a un `SourceBlock`. Puede estar conectada a una `TargetConnection` de otro `BlockModel`.
- **Input:** Define una entrada dentro de un bloque, que puede contener campos o una conexión para otro bloque.
 - **Atributos Clave:** `Name`, `Type` (`EConnection` como `InputValue`, `NextStatement` o `DummyInput`), `Connection` (si no es `Dummy`), `FieldRow` (lista de `FieldModels`), `Align`.
 - **Relaciones:** Pertenece a un `BlockModel`. Su `Connection` puede enlazarse a la `OutputConnection` o `PreviousConnection` de otro `BlockModel`.
- **Field:** Representa un campo individual dentro de un `InputModel` (ej. etiqueta, entrada de texto, desplegable).
 - **Atributos Clave:** `Name`, `Type` (ej. `field_label`, `field_number`), valor textual (`mText`), `IsEditable`.
 - **Relaciones:** Pertenece a un `InputModel` (a través de su `FieldRow`).
- **DataStruct:** Estructura utilizada por el sistema de ejecución (`CSharpInterpreter`) para representar los diferentes tipos de datos que los bloques pueden manejar o devolver (booleano, número, string, lista).

Persistencia de datos (diseño lógico)

El diseño de persistencia para el prototipo se basa en la serialización del `Workspace` completo a un formato **XML**.

- **Estructura XML:** La estructura del XML generado refleja la jerarquía de los bloques, sus inputs, fields, conexiones y valores. Los bloques *TopBlocks* son los elementos raíz bajo la etiqueta `<xml>`. Los bloques anidados aparecen dentro de las etiquetas `<value>`, `<statement>` o `<next>` de sus bloques padres.

- **Datos Serializados:** Se serializan el tipo de bloque, su ID, posición (x, y), estado (colapsado, deshabilitado), valores de los campos, y la estructura de conexión con otros bloques.
- **Mecanismo de Almacenamiento:** Para el prototipo, el XML resultante se almacena localmente utilizando las `PlayerPrefs` de Unity, gestionado por la clase `WorkspaceSerializer`.

Este diseño de datos y persistencia, aunque simple para el prototipo, establece una base que podría extenderse a sistemas de bases de datos más robustos en el futuro.

C.3. Pivote y diseño del sistema híbrido final

Tras la implementación de la arquitectura de bloques visuales, los desafíos técnicos encontrados en la creación de un motor de layout dinámico para los bloques de control de flujo motivaron un **pivote estratégico**. El nuevo diseño se centra en un sistema híbrido que simplifica la interacción visual y introduce un lenguaje de dominio específico (DSL) textual, manteniendo los principios de separación de responsabilidades.

Diseño de datos del sistema híbrido

La estructura de datos del sistema final se simplifica para gestionar los nuevos conceptos:

- **ChallengeData:** Activo de tipo `ScriptableObject` que encapsula la información de cada desafío: título, descripción detallada, etc.
- **Instruction:** Clase que representa un comando único y parseado. Contiene un `CommandType` (enum) y un `Value` (float). Para el comando repetir, esta clase contiene una `List<Instruction>` anidada.
- **Lista de instrucciones (`List<Instruction>`):** Estructura de datos principal que representa el programa completo del usuario. Es el equivalente a un Árbol de Sintaxis Abstracto (AST) simplificado, generado por el `CommandParser`.

Diseño arquitectónico híbrido

La arquitectura final se compone de un conjunto de gestores que orquestan el flujo de la aplicación.

- **Gestores principales:** Se describe el rol de `GameManager`, `IDE_UIManager` y `ChallengeLoader`.
- **Núcleo de interpretación:** Se detalla el `CommandParser` como el componente de análisis y el `RobotController` como el motor de ejecución.

C.4. Diseño procedimental del sistema híbrido

Procedimiento: resolver un desafío

Este caso de uso encapsula el flujo completo del usuario en el prototipo final.

1. **Selección del desafío:** El usuario selecciona un desafío en el `MenuScene`. El `ChallengeLoader` persiste esta selección.
2. **Carga del entorno:** El `GameManager` en la `CodingScene` lee el desafío desde el `ChallengeLoader` y configura la UI (`IDE_UIManager`) con las instrucciones.
3. **Construcción del programa textual:** El usuario escribe comandos en el `InputField`. El `IDE_UIManager` y el `ProgramBuilder` los recopilan y los muestran visualmente como una lista.
4. **Ejecución y parseo:** Al pulsar ejecutar, el `GameManager` envía el código fuente completo al `CommandParser`.
 - **Flujo alternativo (error de sintaxis):** Si el parser detecta un error, se lo comunica al `IDE_UIManager`, que muestra un mensaje al usuario. La ejecución se cancela.
5. **Simulación del robot:** Si el parseo es exitoso, el `GameManager` pasa la `List<Instruction>` al `RobotController`. Este inicia la corrutina `ProcessInstructionList` para ejecutar las acciones en el entorno 3D.

6. **Fin de la ejecución:** La simulación termina (éxito, colisión, etc.) y el RobotController notifica al GameManager, que a su vez instruye a la UI para mostrar el resultado final.

C.5. Diseño arquitectónico

La arquitectura del sistema se ha basado en el patrón **Modelo-Vista-Controlador (MVC)**, adaptado al entorno de desarrollo de Unity y a la estructura de la librería UBlockly.

Diagrama de arquitectura general (MVC) primera versión

Como se detalló en el Capítulo 3 (Conceptos Teóricos, subsección *arquitectura general (Modelo-Vista-Controlador - MVC)*), el sistema se dividió en tres capas principales.

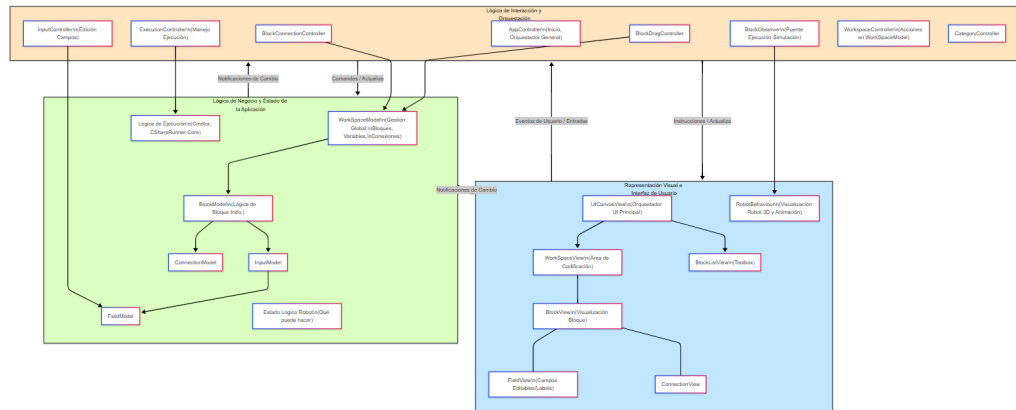


Figura C.1: Visión general de la arquitectura MVC del prototipo - primera versión.

A continuación, se profundiza en los componentes clave de cada capa y sus interacciones.

Componentes clave por capa MVC

- Capa modelo (core UBlockly y lógica de simulación):

- **Workspace, Block, Connection, Input, Field:** Responsables del estado y la lógica de los bloques, heredados y adaptados de UBlockly.
- **CSharpInterpreter, Cmdtor, CmdEnumerator:** Componentes del sistema de ejecución de UBlockly, encargados de la interpretación de los bloques.
- **DataStruct:** Representación de tipos de datos en tiempo de ejecución.

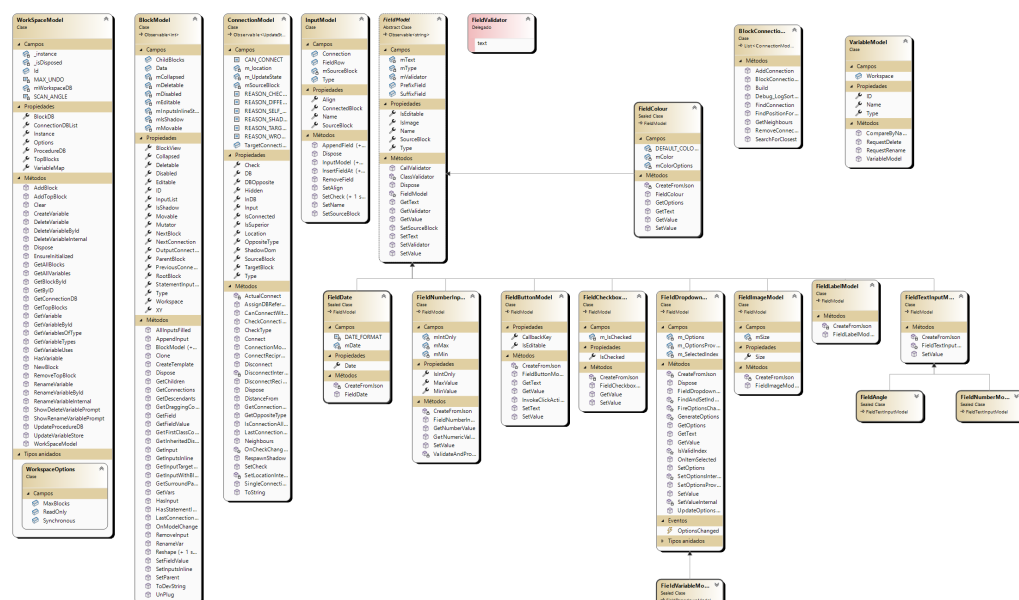


Figura C.2: Diagrama de la capa modelo.

- Capa vista (Unity UI y entorno 3D):

- **UICanvasView**: Orquestador principal de la interfaz de usuario, gestiona los paneles y la transición entre modos.
- **BlockListView**: Vista de la toolbox, muestra categorías y plantillas de bloques.
- **WorkSpaceView**: Vista del área de codificación, contiene y gestiona las **BlockViews** activas.

- [illegible]

Figura C.3: Diagrama de clases de la capa vista.

- **Capa controlador (Gestión de interacción y lógica de aplicación):**
 - **AppController:** Punto de entrada y orquestador general de la aplicación, inicializa otros controladores.
 - **CategoryController:** Maneja la selección de categorías en la Toolbox.
 - **WorkspaceController:** Gestiona las operaciones a alto nivel sobre el `WorkSpaceModel` (ej. añadir/eliminar bloques, solicitar guardado/carga).
 - **BlockDragController:** Implementa la lógica de arrastrar bloques, incluyendo el cálculo de offsets y la interacción con el `DragLayer`.
 - **BlockConnectionController:** Se encarga de la lógica de detección de conexiones válidas (snap) y la gestión del resaltado.
 - **InputController:** Procesa la entrada del usuario en los campos editables de los bloques.
 - **ExecutionController:** Inicia y detiene la ejecución del programa, comunicándose con `CSharpRunner`.
 - **BlockObserver:** Actúa como un intermediario entre el sistema de ejecución de bloques y la representación/acción en la Vista 3D (`RobotBehaviour`).



La comunicación entre las capas sigue el patrón Observador y las interacciones directas de los controladores.

- Un evento de UI (ej. clic en un `FieldView`, arrastre de un `BlockView`) es capturado por el componente vista correspondiente.

- La vista notifica al controlador pertinente (ej. `InputController`, `BlockDragController`).

2. Procesamiento en el controlador:

- El controlador interpreta la acción del usuario.
- Realiza validaciones si es necesario.
- Solicita modificaciones o consultas al modelo (ej. `WorkspaceController` pide al `WorkSpace` que conecte dos `Blocks`).

3. Actualización del modelo:

- El modelo actualiza su estado interno (ej. `Connection.Connect()`).
- Si el estado cambia, el modelo dispara un evento a sus observadores (`BlockModel.FireUpdate()`).

4. Actualización de la vista:

- Las vistas suscritas (ej. `BlockView`, `ConnectionView`) reciben la notificación del modelo.
- Las vistas se actualizan para reflejar el nuevo estado del modelo (ej. `ConnectionView` se redibuja para mostrar una conexión establecida, `BlockView` recalcula su layout).

5. Ejecución del programa:

- El `ExecutionController`, al recibir la orden de ejecutar, pasa el `WorkSpace` al `CSharpRunner`.
- `CSharpRunner` itera sobre los bloques (`CmdEnumerator`) y sus intérpretes (`Cmdtor`).
- Los `Cmdtors` pueden leer datos del modelo (ej. valores de campos) y, a través del `BlockObserver`, enviar comandos a la Vista 3D (`RobotBehaviour`) para la simulación.

C.6. Diseño del sistema de programación híbrido final

Ampliación del diseño de datos: Instrucciones y desafíos

Además de las entidades del modelo de bloques, la arquitectura final introduce las siguientes estructuras de datos clave:

- **ChallengeData:** Activo de tipo `ScriptableObject` que encapsula la información de cada desafío (título, descripción detallada). Este diseño permite crear y modificar el contenido educativo sin necesidad de recompilar la aplicación. Un gestor singleton (**ChallengeLoader**) se encarga de su persistencia entre escenas.
- **Instruction:** Clase que representa un comando único y parseado del DSL. Contiene principalmente un `CommandType` (enum que define la acción, ej. `MoveForward`, `TurnLeft`) y un `Value` (float para parámetros como distancia o grados). Para comandos de control de flujo como ‘repetir’, esta clase contiene una `List<Instruction>` anidada, creando así una estructura de árbol que representa el programa.

Diseño arquitectónico híbrido

La arquitectura final integra el sistema de bloques visual con el nuevo sistema textual. Mientras que los bloques se gestionan mediante el patrón MVC descrito previamente, el flujo de programación textual y la ejecución final son orquestados por un conjunto de gestores simplificados:

- **GameManager:** Controlador central de la escena de simulación. Orquesta el flujo completo: recibe la orden de ejecución desde la UI, invoca al parser y, si es exitoso, pasa la lista de instrucciones al `RobotController`.
- **IDE_UIManager:** Actúa como la Vista y Controlador de la interfaz de programación textual. Captura la entrada de texto, gestiona la visualización de comandos y comunica las acciones del usuario al `GameManager`.
- **CommandParser:** Componente estático que implementa la lógica del intérprete de DSL. Su única responsabilidad es transformar una cadena de texto en una `List<Instruction>` validada.

- **RobotController:** Ejecuta la `List<Instruction>` mediante corrutinas, interactuando directamente con el componente ‘Rigidbody’ del robot y el entorno 3D.
- **BlockController:** Lleva cabo la sincronización de los modelos con las vistas para la creación de los bloques sustituyendo con ello a los controladores de la primera versión. También gestiona el sistema de drag and drop entre bloques.
- **WorkspaceController:** Sustituye a los controladores iniciales que se encargaban de sincronizar los distintos espacios de trabajo.
- **ToolBoxController:** Permite la instanciación de los paneles a implementar para la correcta replica del espacio de trabajo y sus cajas de herramientas para gestionar las categorías y bloques.

C.7. Diseño procedimental

El diseño procedimental se centra en describir el flujo de control para las operaciones más significativas del sistema. A continuación, se detallan algunos procedimientos clave:

Procedimiento: inicialización de la interfaz principal - primera versión

Al iniciar la aplicación, el sistema debe construir y presentar la interfaz de usuario principal. Este proceso es orquestado principalmente por la clase `UICanvasView` durante su ciclo de vida `Awake()` o `Start()`.

1. **Carga de configuraciones fundamentales:** La `UICanvasView` invoca al `BlockResMgr` para cargar configuraciones esenciales, como el archivo XML del `DefaultToolBox.xml` (que define las categorías y bloques de la toolbox) y los archivos de internacionalización (i18n). A partir de esta información, se inicializa el objeto `ToolboxConfig` y se cargan los colores asociados a cada categoría de bloques.
2. **Creación del canvas Principal:** Se instancia el `GameObject` raíz del Canvas de Unity UI, junto con sus componentes esenciales (`Canvas`, `CanvasScaler`, `GraphicRaycaster`), configurándose para el modo *Screen Space Overlay*.

3. **Construcción de paneles de UI:** La `UICanvasView` procede a crear jerárquicamente los diferentes paneles que componen la interfaz:

- *Panel superior de control:* Se instancia el panel superior, añadiéndole el logo de la aplicación y los iconos interactivos (Bandera Verde, Stop, Guardar, Cargar).
- *Panel izquierdo (Categorías):* Se crea el panel para los botones de selección de categorías, equipado con un `ScrollRect` y un contenedor con `VerticalLayoutGroup` para albergar los botones de categoría que se generarán posteriormente.
- *Paneles centrales (Workspace):* Se crea un panel intermedio (`WorkArea`) que a su vez contiene:
 - El Panel de Toolbox (Centro-Izquierda, `BlockListPanelRect`), donde se instanciará `BlockListView` y que incluye un `ScrollRect` para la lista de bloques.
 - El Área de codificación (Derecha, `CodingAreaPanelRect`), donde se instanciará `WorkspaceView`.
- *Capa de arrastre (DragLayer):* Se instancia un panel transparente superpuesto al área de codificación y la toolbox, destinado a ser el padre temporal de los bloques durante las operaciones de arrastre.
- *Paneles de mensajes de simulación:* Se crean los `GameObjects` para el texto de cuenta atrás y los mensajes del robot, inicialmente ocultos.

4. **Instanciación del modelo principal:** Se crea la instancia del `Workspace`, que actuará como el contenedor lógico de todos los bloques y el estado del programa.

5. **Disponibilidad de la interfaz:** La `UICanvasView` marca su inicialización como completa (`m_isCoreComponentsReady = true`), permitiendo que otros controladores (`AppController`) procedan con la vinculación de modelos y la inicialización de funcionalidades más específicas (como la carga de la primera categoría en la Toolbox).

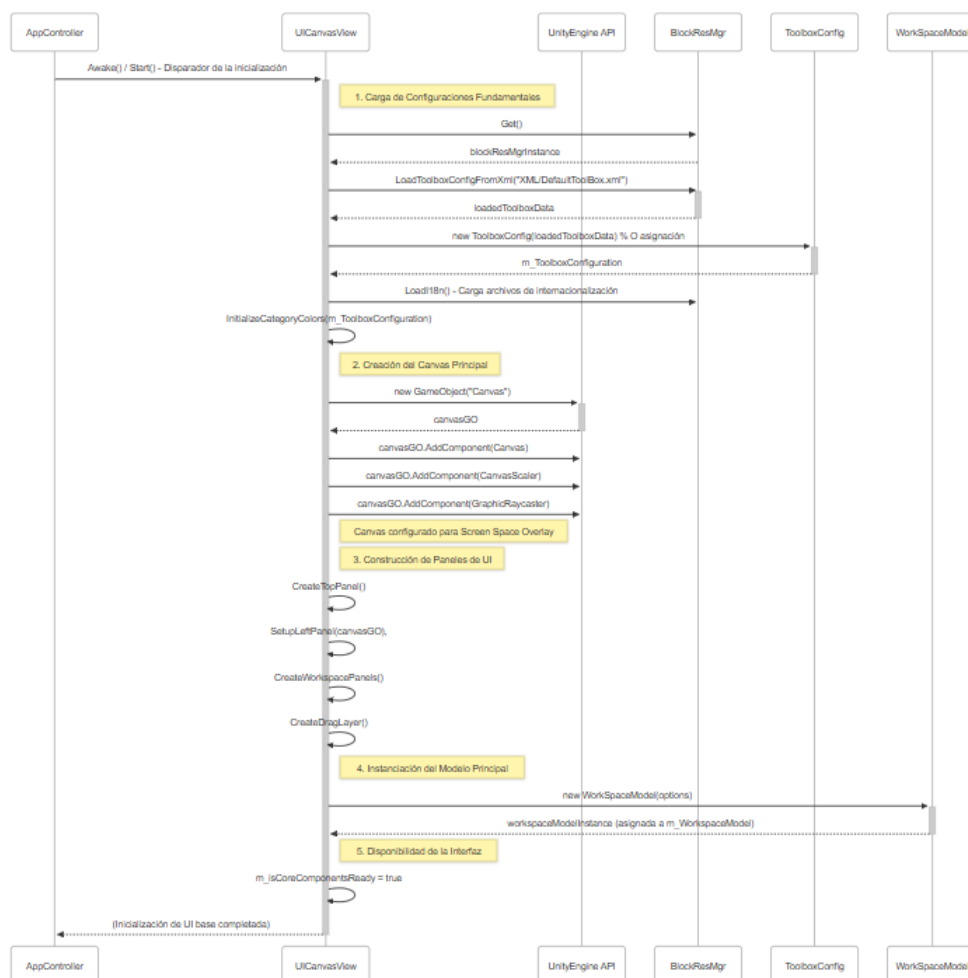


Figura C.5: Diagrama de secuencia: Inicialización interfaz principal.

Procedimiento: selección de categoría y carga de bloques en toolbox

Una vez la interfaz está inicializada, el usuario puede interactuar con el panel de categorías para visualizar los bloques disponibles.

1. El Estudiante pulsa un botón de categoría en el **Panel de Categorías** de la **UICanvasView**.
2. El evento de clic notifica al **CategoryController**.

3. El `CategoryController` invoca el método `ShowBlockCategory` en la `BlockListView` (Toolbox), pasándole el nombre de la categoría seleccionada y su color.
4. La `BlockListView` primero limpia cualquier plantilla de bloque previamente mostrada (`ClearBlockTemplates()`).
5. Luego, la `BlockListView` accede a la `ToolboxConfig` para obtener la lista de tipos de bloques asociados a la categoría seleccionada.
6. Para cada tipo de bloque en la lista:
 - Se invoca a `BlockFactory.CreateBlock(null, blockType)` para crear un `BlockModel` plantilla (sin asignarle un workspace).
 - Se invoca a `BlockViewFactory.CreateView(templateModel, blockListViewInstance)` para instanciar el prefab de la `BlockView` correspondiente a esa plantilla.
 - La `BlockView` plantilla se configura (`InToolbox = true`, color, se le añade un `BlockTemplateDragSource`) y se añade como hijo visual del contenedor de la `BlockListView`.
7. Finalmente, la `BlockListView` fuerza una actualización de su layout y ajusta el scroll para mostrar los nuevos bloques.

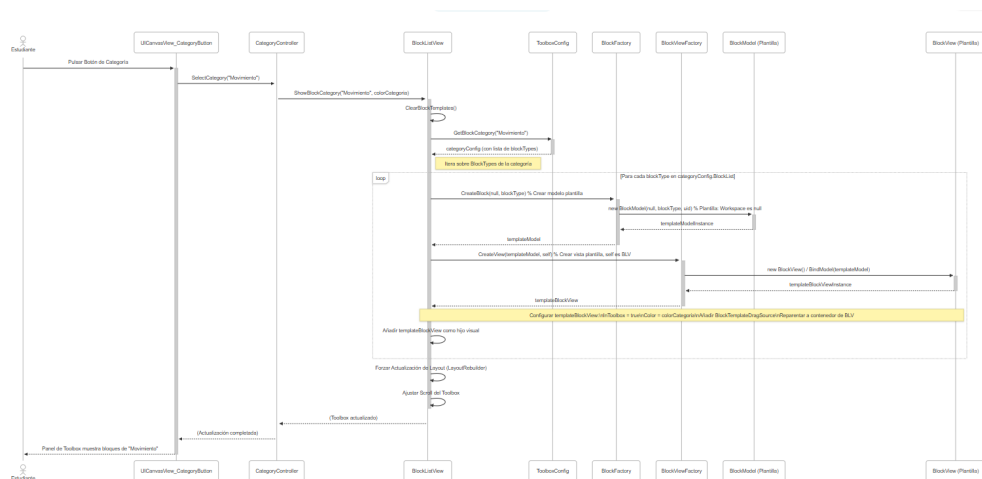


Figura C.6: Diagrama de secuencia: Selección de categoría y visualización de bloques en toolbox.

Procedimiento: Arrastre y conexión de un bloque (*Drag and Snap*)

1. Inicio del arrastre (OnBeginDrag en BlockView o BlockTemplateDragSource):

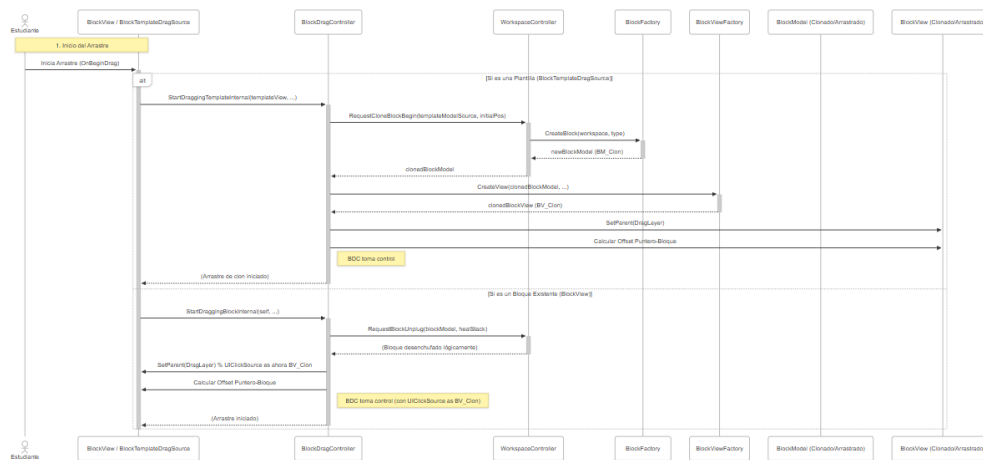


Figura C.7: Diagrama de Secuencia: Inicio del arrastre de un bloque.

- Si es una plantilla, se clona el **BlockModel** y se crea su **BlockView** asociada (**BlockFactory**, **BlockViewFactory**).
- El **BlockDragController** toma el control. El **BlockView** se reparenta al **DragLayer**.
- Se calcula un offset entre el puntero y el pivote del bloque.

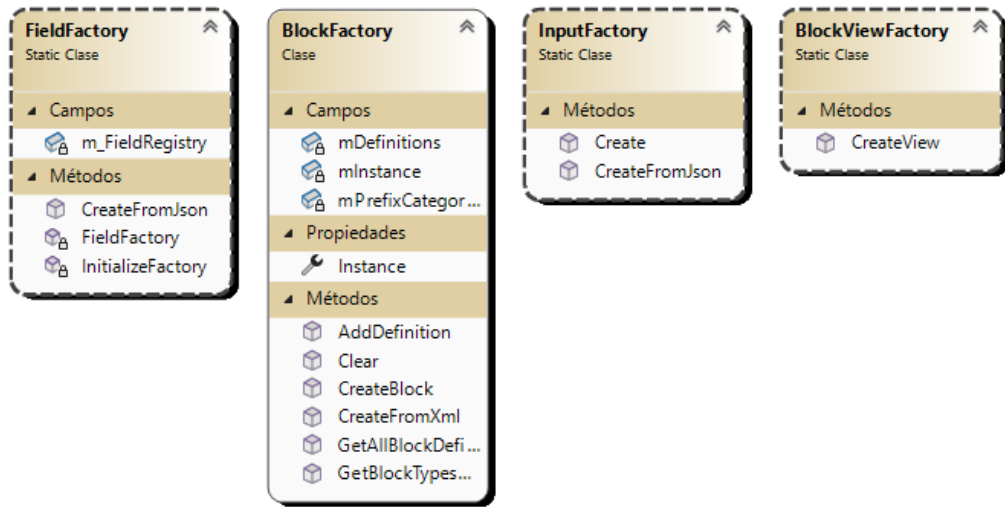


Figura C.8: Diagrama de clases constructores - factory.

2. Durante el arrastre (`OnDrag` -> `BlockDragController.HandleDrag()`):

- La posición del `BlockView` se actualiza siguiendo el puntero más el offset.
- El `BlockConnectionController.ProcessDrag()` es invocado:
 - Itera sobre las conexiones del bloque arrastrado (`BlockModel.GetDraggingConnections()`).
 - Para cada conexión, busca en la `BlockConnectionDB` opuestas las conexiones candidatas cercanas dentro de un radio (`SearchForClosest()`).
 - Valida si la conexión es permitida (`ConnectionModel.IsConnectionAllowed()`).
 - Si hay un candidato válido, la `ConnectionView` del bloque estático activa su resaltado (`Highlight(true)`). Si el mejor candidato cambia, el resaltado anterior se desactiva.

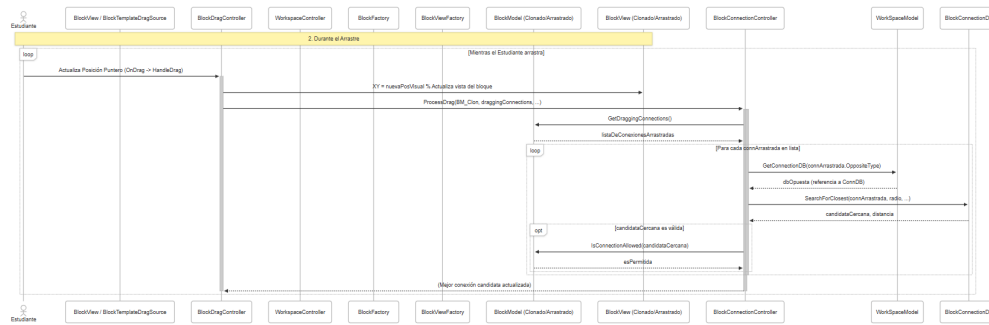


Figura C.9: Diagrama de Secuencia: Durante del arrastre de un bloque.

3. Fin del arrastre (`OnEndDrag -> BlockDragController.HandleEndDrag()`):

- El `BlockConnectionController` finaliza la búsqueda y determina la mejor pareja de conexiones.
- Si se encuentra una conexión válida (*snap*):
 - Se invoca `Connection.Connect()` en los modelos lógicos. Esto puede disparar la lógica de bumping para desplazar bloques existentes.
 - La `BlockView` del bloque arrastrado (o del bloque desplazado) se reparenta al contenedor visual apropiado del bloque padre (ej. `nextStatementContainer` o un `InputView`).
 - Se actualiza la posición (`anchoredPosition`) de la `BlockView` conectada para el encaje visual.
 - Se actualiza el layout de los bloques afectados.
- Si no hay conexión válida, se gestiona la colocación en el espacio de trabajo o el descarte (`BlockConnectionController.HandleDropPlacement()`):
 - Si se suelta en el área de codificación, el `BlockModel` se registra en el `Workspace` (si es un clon nuevo) o su posición se actualiza. El `BlockView` se reparenta al `CodingAreaPanelRect`.
 - Si se suelta fuera de un área válida, el bloque se elimina (`Block.Dispose()` y `BlockView.Dispose()`).
- Se resetea el estado de arrastre y los resaltados.

- El `CmdRunner` inicia una corrutina (`Run()`) que procesa su pila de `CmdEnumerators`.
- En cada paso, toma el `CmdEnumerator` actual, obtiene su `Cmdtor` asociado (intérprete) e invoca su método `Run(block)`.
- El `Cmdtor.Run()` (que devuelve un `IEnumerator`) se ejecuta.

4. Interpretación del bloque (`Cmdtor`):

- **Ejemplo `MotionBlockInterpreter`:**
 - Obtiene los parámetros del `Block` (ej. valor del input `STEPS` usando `CSharp.Interpreter.ValueReturn()`).
 - Invoca un método en el `BlockObserver` (ej. `m_BlockObserver.MoveRobot(stepsVal`
 - Realiza un `yield return` sobre la corrutina devuelta por el `BlockObserver`, pausando la ejecución del intérprete del bloque actual hasta que la acción del robot termine.

5. Interacción con la simulación (`BlockObserver` y `RobotBehaviour`):

- `BlockObserver.MoveRobot()` obtiene la referencia al `GameObject` del robot.
- Invoca una corrutina en el `RobotBehaviour` (ej. `robotBehaviour.MoveRobotOverTime()`)
- Esta corrutina del `RobotBehaviour` anima el movimiento del robot en la escena 3D a lo largo del tiempo (ej. usando `Vector3` y `yield return null`).

6. Continuación del flujo y Finalización:

- Una vez que la corrutina del `Cmdtor` (y por ende la del `BlockObserver` y `RobotBehaviour`) finaliza, el `CmdRunner` obtiene el siguiente `CmdEnumerator` de la pila (`CmdEnumerator.GetNextCmd()`) y repite el proceso.
- Si no hay más bloques o se invoca `ExecutionController.StopExecution()`, la ejecución finaliza.

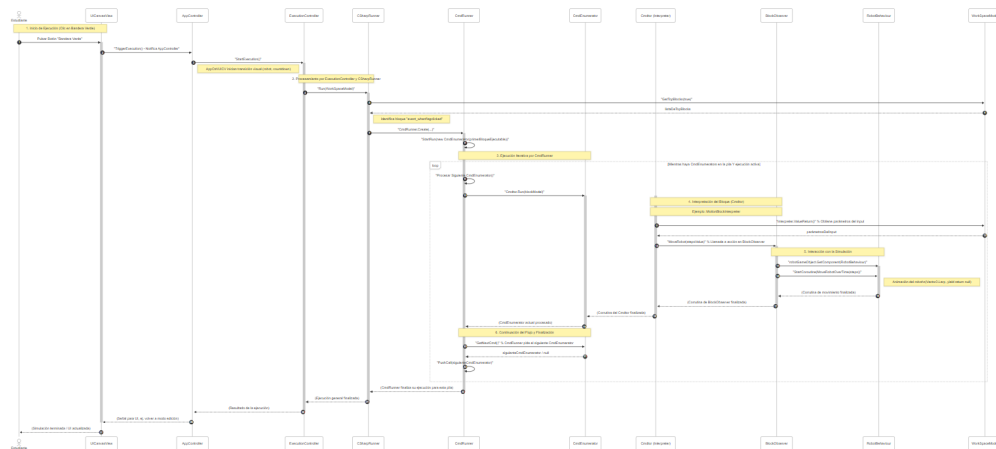


Figura C.11: Diagrama de secuencia: Ejecución de un programa de bloques

Diseño procedimental del sistema híbrido

A continuación se detalla el procedimiento clave del sistema híbrido.

Procedimiento: Resolver un desafío

Este caso de uso encapsula el flujo completo del usuario en el prototipo final, integrando la selección del desafío, la construcción del programa y su simulación.

1. Selección y carga del desafío:

- El usuario selecciona un desafío en el MenuScene.
- ChallengeLoader almacena el ChallengeData seleccionado.
- Al cargar la CodingScene, el GameManager obtiene el desafío del ChallengeLoader y se lo comunica al IDE_UIManager.
- El IDE_UIManager muestra las instrucciones y el título del desafío.

2. Construcción del programa textual:

- El usuario introduce un comando del DSL en el InputField.

- Al pulsar añadir (+), el IDE_UIManager añade el comando a su ‘ProgramBuilder’ interno y crea un bloque de texto visual para feedback.

3. Ejecución y simulación:

- El usuario pulsa ejecutar. El IDE_UIManager notifica al GameManager.
- El GameManager solicita el código completo al ProgramBuilder y se lo pasa al CommandParser.Parse().
- Si el Parse() falla, devuelve un mensaje de error que el GameManager envía a la UI.
- Si el Parse() tiene éxito, devuelve una List<Instruction>. El GameManager pasa esta lista al RobotController.ExecuteProgram().
- El RobotController inicia una corrutina que itera sobre la lista, ejecutando cada instrucción (y sus sub-listas anidadas) de forma secuencial y esperando a que termine cada acción.
- Durante la ejecución, el RobotController realiza SweepTest‘ç para la detección proactiva de colisiones. Si detecta una, detiene su corrutina y notifica al GameManager.
- Al finalizar (por éxito o colisión), el GameManager informa el resultado al IDE_UIManager.

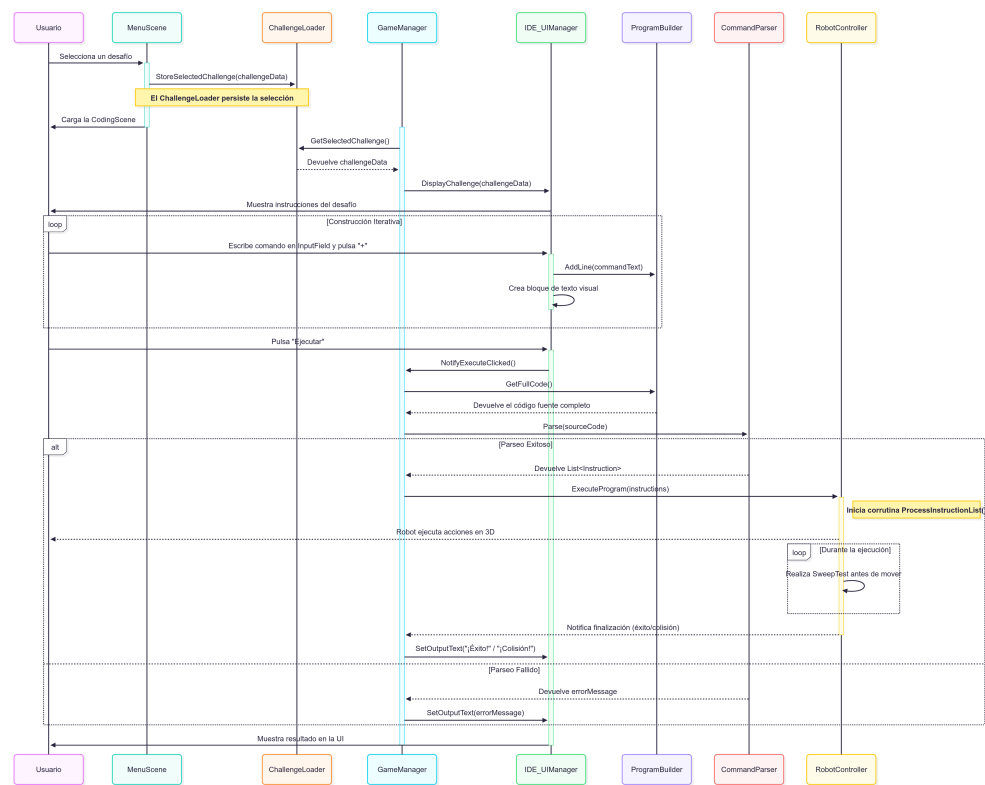


Figura C.12: Diagrama de secuencia: Flujo de ejecución del programa textual.

Evolución arquitectónica: De bloques a híbrido

El pivote estratégico hacia un sistema híbrido conllevó una significativa refactorización de la capa de Controladores. Las responsabilidades originalmente distribuidas en múltiples clases para gestionar la interacción visual se consolidaron en gestores más específicos para el nuevo flujo de trabajo. La siguiente tabla resume la evolución de los principales componentes de control:

Responsabilidad	Componente en arquitectura inicial (Bloques - MVC)	Componente en arquitectura final (híbrido)
Orquestación general de la aplicación	AppController (Inicializa todos los sistemas y maneja estados globales como Play/Stop).	GameManager (Orquesta la escena de desafío y ejecución) y ChallengeLoader (Gestiona el flujo entre escenas). BlocController (Gestiona las relaciones entre el modelo de datos y la vista).
Creación de programas (Lógica del usuario)	Interacción directa del usuario con las vistas de los bloques, gestionada por varios controladores: ▪ ToolboxController: Maneja la creación de bloques desde la Toolbox. ▪ BlockController: Gestiona el arrastre y las conexiones de un bloque ya en el workspace.	La lógica de creación recae en dos flujos distintos: ▪ Textual: IDE_UIManager captura el texto y ProgramBuilder lo acumula. ▪ Visual (para estructura): La lógica de arrastrar y soltar se mantiene para bloques estructurales, simplificando la necesidad de múltiples controladores.
Procesamiento del Programa	El programa se define por la estructura de Blocks interconectados. No hay una fase de parseo explícita.	CommandParser: Realiza el análisis léxico y sintáctico del DSL textual, generando una lista de Instruction (el AST del programa).
Ejecución del Programa	ExecutionController que activa el CSharp.Runner . Este último recorre el árbol de bloques y ejecuta los intérpretes ('Cmdtor') correspondientes.	RobotController: Recibe la lista de instruction y la ejecuta de forma secuencial mediante corrutinas, sin necesidad de la pila de ejecución de UBlockly.

Tabla C.1: Tabla comparativa de la evolución de componentes arquitectónicos.

Apéndice D

Documentación Técnica de Programación

D.1. Introducción

A continuación se proporciona la documentación técnica esencial para comprender, compilar, ejecutar y extender el prototipo del laboratorio virtual de programación y robótica que se ha desarrollado en el presente TFG. Este documento está dirigido a desarrolladores o evaluadores que necesiten interactuar con el código fuente del proyecto y quieran seguir contribuyendo a su definición. En él se detalla la estructura de directorios, las clases principales y sus responsabilidades, las instrucciones para la compilación y ejecución, y un resumen de las pruebas realizadas para validar el sistema.

El proyecto ha sido desarrollado en el motor de videojuegos **Unity (Versión 6000.0.2f1)** utilizando el lenguaje de programación **C#** codificado en el IDE visual studio. La interfaz de programación por bloques se basa en una adaptación de la librería de código abierto **UBlockly** [?] la cual ha sido modificada y adaptada al patrón MVC.

D.2. Estructura de directorios del proyecto primera versión

El proyecto en Unity sigue una estructura de directorios organizada para facilitar la navegación y la gestión de los diferentes tipos de activos y scripts.

A continuación, se describe la organización de las carpetas principales dentro del directorio **Assets/**:

- **Assets/Core/**: Contiene las clases fundamentales de la librería UBlockly que han sido directamente integradas o adaptadas, así como definiciones esenciales.
 - **Code/**: Lógica central de UBlockly para la interpretación (**Cmdtor**, **CmdEnumerator**, **CmdRunner**) y generación de código (**Generator**), así como estructuras de datos (**DataStruct**, **Datas**).
 - **CustomDefine/**: Clases y enumeraciones personalizadas o fundamentales para el proyecto, como la estructura **Number**.
 - **Field/**: Modelos de campos (**FieldModel**) que no requerían un diálogo UI propio o que son extensiones de UBlockly.
 - **Mutator/**: Lógica para los mutadores de bloques (aunque no se hayan implementado extensivamente en el prototipo).
 - **Variable/**: Clases para la gestión de variables (**VariableMap**, **VariableModel**).
- **Assets/CSharp/**: Clases específicas para la implementación en C# de la lógica de UBlockly.
 - **Generators/**: (Heredado de UBlockly, no es el foco principal de este TFG, que se centra en la interpretación directa).
 - **Interpreters/**: Clases intérprete (**Cmdtor**) para cada tipo de bloque ejecutable. Aquí reside **MotionBlockInterpreter.cs**. Contiene **CSharpInterpreter.cs** que mapea tipos de bloques a sus intérpretes.
 - **CSharp.cs**, **CSharpGenerator.cs**, **CSharpInterpreter.cs**, **CSharpRunner.cs**. Clases centrales del sistema de UBlockly para C#.
- **Assets/Execute/**: Scripts relacionados directamente con la ejecución y simulación.
 - **BlockObserver.cs**: Puente entre la ejecución de bloques y el robot en la simulación.

- `MotionBlockInterpreter.cs`: (Originalmente lo tenías aquí, pero conceptualmente encaja mejor en `Assets/CSharp/Interpreters/`. Asegúrate de dónde está realmente).
- `RobotBehaviour.cs`: Script que controla el comportamiento y animación del `GameObject` del robot.
- **Assets/Interfaces/**: Definiciones de interfaces C# utilizadas en el proyecto (ej. `IBlockElement`, `IObserver`).
- **Assets/Resources/**: Carpeta especial de Unity para activos que se cargan dinámicamente.
 - `XML/Blocks/`: Archivos XML que definen la estructura de los bloques de cada categoría (ej. `movimiento.xml`, `eventos.xml`).
 - `XML/`: Archivos XML de configuración general (ej. `Categories.xml`, `DefaultToolBox.xml`).
 - `Icons/`: Imágenes para los iconos de la interfaz (bandera verde, stop, etc.) y posiblemente para los bloques.
 - `BlockResSettings.asset`: `ScriptableObject` que gestiona la carga de recursos como las definiciones de bloques y prefabs (heredado/adaptado de `UBlockly`).
 - `Prefabs/`: Contiene elementos visuales de Unity para la disposición de vistas como el `CategoryButtonPrefab` usado para representar el círculo de las categorías.
 - `Prefabs/Blocks/`: Contiene el diseño de los prefabs que representan los distintos tipos de bloques. Inicialmente se han creado los siguientes prefabs
 - `event_whenflagclicked` que representa al bloque al hacer click en bandera verde.
 - `Highlight` que representa a resaltado que se muestra cuando se van a conectar dos bloques.
 - `motion_movesteps` que representa el bloque mover 10 pasos.

Es en esta carpeta de prefabs donde se tienen que incluir los diferentes bloques que se diseñen para cada una de las categorías que hay.

- **Assets/Scripts/:** Contiene la mayoría del código C# desarrollado específicamente para la lógica y la interfaz del prototipo.
 - **Blocks/Controller/:** Clases Controlador del patrón MVC (`AppController.cs`, `WorkspaceController.cs`, `BlockDragController.cs`, `BlockConnectionController.cs`, `ExecutionController.cs`, `InputController.cs`, `CategoryController.cs`).
 - **Blocks/DB/:** Clases relacionadas con la base de datos de conexiones (`BlockConnectionDB.cs`). (Tu clase estaba aquí)
 - **Blocks/:** Clases auxiliares o generales del sistema de bloques (`BlockDataLoader.cs`, `CategoryLoader.cs`, `WorkspaceSerializer.cs`, etc.).
 - **Definition/:** Clases que definen la estructura de los bloques y argumentos (`ArgumentDefinition.cs`, `BlockDefinition.cs`).
 - **Model/BlocksModels/:** Clases Modelo de UBlockly y la adaptación (`BlockModel.cs`, `ConnectionModel.cs`, `InputModel.cs`).
 - **Model/FieldModels/:** Clases para los modelos de campos (`FieldLabelModel.cs`, `FieldNumberInputModel.cs`, etc.).
 - **View/FieldViews/:** Componentes `MonoBehaviour` para la vista de los campos (`FieldLabelView.cs`, `FieldInputView.cs`, etc.).
 - **View/:** Componentes `MonoBehaviour` para las vistas principales (`BlockView.cs`, `ConnectionView.cs`, `InputView.cs`, `LineGroupView.cs`, `BlockListView.cs`, `WorkSpaceView.cs`, `UICanvasView.cs`).
 - **Observables/:** Implementación del patrón Observador.
 - **Types/:** Enumeraciones y tipos de datos auxiliares.
 - **UI/:** Prefabs para elementos de UI como botones de categoría, etc.
 - **Robot/:** Prefab del modelo 3D del robot.
- **Assets/Scenes/:** Escenas de Unity utilizadas en el proyecto (ej. la escena principal del laboratorio).

Esta estructura busca separar claramente la lógica del núcleo de UBlockly, las adaptaciones específicas para C# y Unity, el código de la aplicación (MVC), y los recursos y configuraciones.

D.3. Estructura de directorios del proyecto segunda versión

Assets/Scripts/: Contiene la mayor parte del código C# del prototipo.

- **Desafios/:** Scripts centrales que gestionan el flujo de la aplicación híbrida: `GameManager.cs`, `IDE_UIManager.cs`, `RobotController.cs`, `CommandParser.cs`, `Instruction.cs`, y el sistema de gestión de desafíos con `ChallengeData.cs` y `ChallengeLoader.cs`.
- **Scenes/:** Scripts específicos para las escenas del menú principal (`MenuManager.cs`, `SceneNavigator.cs`).
- **Debugger/:** Herramientas de depuración personalizadas, como el analizador de prefabs.
- **Logger.cs:** Clase estática para la gestión centralizada de logs.

Assets/Source/: Contiene las clases fundamentales de la librería UBlockly que fueron adaptadas y sirvieron como base para el sistema de bloques visuales.

- **Core/:** Lógica central del modelo de datos de UBlockly (`Block.cs`, `Connection.cs`, `Workspace.cs`, `Field.cs`, etc.) y el motor de ejecución (`CSharpRunner.cs`, `Cmdtor.cs`). Se incluye en este directorio el de controllers para la integración del MVC.
- **CodeDB/:** Intérpretes (interpreters) y generadores (generators) originales de UBlockly para distintos tipos de bloques.
- **UGUIView/:** Componentes de vista (`BlockView.cs`, etc.) y de la interfaz de usuario (Dialogs, Toolbox) de la implementación original de UBlockly.

Assets/Resources/: Carpeta especial de Unity para activos cargados dinámicamente.

- **Challenges/:** Contiene los activos ScriptableObject (`.asset`) para cada uno de los desafíos definidos.
- **Icons/:** Imágenes utilizadas en la interfaz de usuario (bandera verde, logo, etc.).

- **Prefabs/:** Prefabs de UI y otros elementos reutilizables.
 - **CommandBlockPrefab.prefab:** Prefab para representar visualmente cada línea de código en la IDE textual.
 - **ChallengeCardPrefab.prefab:** Plantilla para las tarjetas de desafío en el menú.
 - *(Y cualquier otro prefab clave de tu MVP)*

Assets/Scenes/: Contiene las escenas de Unity: **MenuScene** (menú principal y selección de desafíos) y **CodingScene** (entorno de programación y simulación).

Esta estructura distingue claramente el código del prototipo final, las librerías base utilizadas, y los recursos artísticos y de configuración.

D.4. Manual del programador (clases y módulos principales)

Dentro de este punto se ofrece una visión general de las clases más importantes y cómo interactúan, enfocándose en los componentes desarrollados o significativamente adaptados para este TFG.

Arquitectura MVC y flujo de control primera versión

Como se detalló en el apéndice C (especificación de diseño), el sistema sigue un patrón MVC. A continuación, se destacan los roles de las clases controladoras clave:

- **AppController.cs:** Orquesta la inicialización de la aplicación, incluyendo la carga de recursos, la creación de los modelos y vistas principales, y la instanciación de otros controladores. Maneja la transición entre el modo de programación y el modo de simulación.
- **WorkspaceController.cs:** Actúa como la interfaz principal para realizar operaciones sobre el **WorkSpaceModel** (añadir, eliminar, mover bloques). Valida las solicitudes antes de modificar el modelo.
- **BlockDragController.cs:** Gestiona toda la lógica de arrastre de **BlockViews**, incluyendo la clonación desde la **Toolbox**, el reparentado al **DragLayer** y la actualización de posiciones.

- **BlockConnectionController.cs:** Trabaja en conjunto con **BlockDragController** para identificar conexiones válidas, gestionar el resaltado visual y solicitar al **WorkspaceController** que establezca las conexiones lógicas.
- **ExecutionController.cs:** Inicia y detiene la ejecución del programa de bloques, interactuando con el **CSharpRunner** de UBlockly.
- **InputController.cs:** Procesa los cambios de valor en los campos editables de los bloques (**FieldViews**), validándolos y solicitando la actualización del **FieldModel** correspondiente.
- **CategoryController.cs:** Gestiona la lógica de selección de categorías en la **Toolbox** y la actualización de la **BlockListView**.

Sistema de bloques (adaptación de UBlockly)

- **Modelos (Block, Connection, etc.):** Se utiliza la estructura de modelos de UBlockly. La principal adaptación reside en cómo estos modelos son observados y renderizados por las Vistas en Unity UI y cómo interactúan con los controladores.
- **Vistas (BlockView, ConnectionView, etc.):** Son componentes **MonoBehaviour** responsables del renderizado. Se han diseñado prefabs para cada tipo de bloque para asegurar una apariencia consistente con Scratch. El sistema de layout manual implementado en **BaseView** y sus herederos es crucial.
- **Creación de bloques (BlockFactory, BlockDataLoader):** Los bloques se definen en XML y se cargan usando estas clases para crear instancias de **BlockModel** y **BlockDefinition**.

Flujo de control y arquitectura del MVP - sistema híbrido

La arquitectura se organiza en torno a gestores que siguen los principios de separación de responsabilidades:

- **ChallengeLoader.cs:** Utilizando el patrón Singleton, este script persiste la selección del desafío del usuario entre la **MenuScene** y la **CodingScene**, asegurando que el contexto correcto sea cargado.

- **GameManager.cs:** Es el orquestador principal en la CodingScene. Al iniciarse, obtiene el desafío seleccionado del ChallengeLoader, instruye a la UI para que muestre la información, y al recibir la orden de ejecutar, coordina el proceso de parseo y la activación del robot.
- **IDE_UIManager.cs:** Actúa como la capa de Vista-Controlador para la interfaz. Gestiona todos los elementos de UI, captura la entrada de comandos del usuario, la añade a un **ProgramBuilder**, y notifica al **GameManager** cuando el usuario pulsa Ejecutar. También es responsable de mostrar el feedback del sistema (errores, mensajes de éxito, etc.).

Intérprete del DSL (Parser y ejecutor)

- **CommandParser.cs:** Esta clase estática es el corazón del intérprete. Recibe una cadena de texto con el programa completo, la procesa línea por línea, valida la sintaxis de cada comando (mover, girar, repetir), y la traduce en una estructura de datos `List<Instruction>`, que es una representación intermedia y ejecutable del programa. Gestiona los bucles anidados mediante una Pila.
- **Instruction.cs:** Estructura de datos simple que representa un comando individual con su tipo y valor, y opcionalmente, una lista de instrucciones anidadas para los bucles.
- **RobotController.cs:** Recibe la `List<Instruction>` y la ejecuta de forma secuencial mediante corrutinas de Unity. Implementa la lógica de movimiento y giro del robot (`MoveForwardRoutine`, `TurnRoutine`), así como la detección de colisiones mediante `Rigidbody.SweepTest` para un comportamiento proactivo.

Un desarrollador que desee **añadir un nuevo comando al DSL** necesitaría:

1. Añadir un nuevo valor al enum `CommandType`.
2. Extender la lógica del switch-case en `CommandParser.cs` para reconocer la nueva sintaxis.
3. Añadir una nueva corrutina o método en `RobotController.cs` para implementar la acción física del comando.
4. Añadir un nuevo caso en el bucle de ejecución de `ProcessInstructionList` en `RobotController.cs` para invocar la nueva acción.

Sistema de ejecución y simulación

- **CSharpRunner y Cmdtor:** El sistema de ejecución de UBlockly se ha utilizado con mínimas modificaciones en su núcleo. La adaptación principal ha sido la creación de **Cmdtors** específicos como **MotionBlockInterpreter**.
- **BlockObserver.cs:** Actúa como un servicio intermediario. Los **Cmdtors** le solicitan acciones (ej. mover robot) y el **BlockObserver** las traduce en llamadas a corrutinas del **RobotBehaviour.cs**. También gestiona el feedback visual/textual de la ejecución.
- **RobotBehaviour.cs:** Controla el **GameObject** del robot, implementando métodos (a menudo corrutinas) para sus acciones (ej. `MoveRobotOverTime()`).

Un desarrollador que desee añadir un **nuevo tipo de bloque funcional** necesitaría:

1. Añadir su definición en un archivo XML en **Assets/Resources/XML/Blocks/**.
2. Crear un prefab para su **BlockView** en **Assets/Prefabs/Blocks/** (o donde corresponda).
3. (Si es un bloque ejecutable) Crear una nueva clase intérprete que herede de **Cmdtor** en **Assets/CSharp/Interpreters/** y registrarla en **CSharpInterpreter.cs**.
4. (Si el bloque realiza una nueva acción del robot) Añadir el método correspondiente en **RobotBehaviour.cs** y la lógica en **BlockObserver.cs** para invocarlo.

D.5. Compilación, instalación y ejecución del proyecto

Requisitos previos

- **Unity Hub y Unity Editor (Versión 6000.0.2f1 o compatible).**
- Módulo de **Android Build Support** instalado en Unity (si se va a compilar para Android).
- (Opcional) Un dispositivo Android configurado para pruebas o un emulador Android configurado en Android Studio.

- Git, para clonar el repositorio.

Obtención del código fuente

1. Clonar el repositorio desde GitHub:
`git clone [https://github.com/LuisIgnaciodeLunaGomez/VRLearnT0BOT]`
2. Abrir el proyecto clonado con Unity Hub, seleccionando la carpeta raíz del proyecto.

Ejecución en el Editor de Unity

1. Una vez abierto el proyecto en Unity, localizar la escena principal (ej. en `Assets/Scenes/MainMenu.unity`).
2. Abrir la escena principal.
3. Pulsar el botón play en la barra de herramientas superior del editor de Unity.
4. La aplicación se ejecutará en la ventana game del editor.

Compilación y ejecución en dispositivo Android (APK)

1. Conectar el dispositivo Android al ordenador con el modo de depuración USB habilitado.
2. En Unity, ir a **File > Build Settings...**
3. Seleccionar **Android** como plataforma. Si no está activa, pulsar **Switch Platform**.
4. (Opcional) Ajustar las **Player Settings...** (ej. nombre de la compañía, nombre del producto, identificador del paquete, íconos, orientaciones permitidas).
5. Asegurarse de que el dispositivo de prueba esté detectado por Unity (aparecerá en la lista de **Run Device**).
6. Pulsar **Build And Run**. Unity compilará el proyecto, generará un archivo **.apk**, lo instalará en el dispositivo conectado y lo ejecutará.

7. Alternativamente, pulsar **Build** para generar solo el archivo **.apk**, que luego se puede transferir e instalar manualmente en cualquier dispositivo Android compatible.

D.6. Pruebas del sistema

Dada la naturaleza de prototipo y el desarrollo individual de este TFG, las pruebas se han centrado principalmente en la **verificación funcional manual** y la **depuración iterativa** durante el desarrollo de cada característica.

Tipos de pruebas realizadas

- **Pruebas de unidad (conceptuales):** Durante el desarrollo de clases individuales (**Block**, **Connection**, intérpretes, controladores), se realizaron pruebas aisladas de sus métodos principales mediante logs (**Debug.Log**) y el inspector de Unity para verificar que la lógica interna funcionaba como se esperaba antes de su integración. Se diseñó una clase específica llamada **Logger** que permitía ofrecer mayor información en la depuración incluyendo la hora de la ejecución que era fundamental para poder seguir el contenido del archivo **editor.log**.
- **Pruebas de sistema automatizadas (con Katalon Studio):** Para la validación del sistema completo desde la perspectiva del usuario final, se utilizó la herramienta **Katalon Studio**. Se desarrollaron scripts de prueba automatizados para ejecutar casos de uso de extremo a extremo, asegurando la consistencia y la ausencia de regresiones tras cada iteración de desarrollo.

Pruebas de integración (manuales): Tras la implementación de cada nueva funcionalidad que involucraba múltiples componentes MVC (ej. drag and drop, conexión de bloques, ejecución del robot), se realizaron pruebas manuales exhaustivas en el editor de Unity para verificar que los componentes interactuaban correctamente. Se prestó especial atención a:

- La correcta creación y vinculación de modelos y vistas.
- El flujo de eventos entre vistas, controladores y modelos.

- La integridad de las bases de datos de conexiones (`ConnectionDBList`) después de operaciones de arrastre y conexión (verificado con la herramienta de exportación a JSON).
- La correcta traducción de los bloques a acciones del robot.

Pruebas funcionales del prototipo: Se definieron escenarios de prueba clave para validar el flujo principal del laboratorio virtual:

- Escenario 1: Crear un programa simple (Bandera verde -> Mover 10 pasos). Ejecutar. Verificar que el robot se mueve correctamente.
- Escenario 2: Crear un programa con múltiples bloques de movimiento. Ejecutar. Verificar secuencia.
- Escenario 3: Probar la lógica de bumping al insertar bloques en una pila existente.
- Escenario 4: Probar el descarte de bloques para verificar que los mismos eran eliminados del espacio de trabajo.

Pruebas de usabilidad (informales): Se evaluó la facilidad de uso de la interfaz de forma continua durante el desarrollo, intentando adoptar la perspectiva de un usuario final y corrigiendo flujos de interacción confusos o poco intuitivos.

Pruebas en dispositivo Android (básicas): Una vez generados los APKs, se realizaron pruebas en una tablet Android para verificar la funcionalidad básica del arrastre, la ejecución y la visualización de la simulación en la plataforma objetivo.

Herramientas de depuración utilizadas

- **Consola de Unity y Debug.Log:** Herramienta principal para el seguimiento de variables, flujo de control y detección de errores.
- **Inspector de Unity:** Para examinar el estado de los `GameObjects` y sus componentes en tiempo real.
- **Debugger de Visual Studio:** Para el análisis paso a paso del código C# y la inspección de variables en puntos críticos.

- **Herramienta de exportación de ConnectionDB a JSON:** Desarrollada como parte del proyecto para permitir el análisis externo de la coherencia de las bases de datos de conexiones internas de UBlockly.

Aunque no se implementó un framework de pruebas unitarias automatizadas (como NUnit o Unity Test Framework) debido a las limitaciones de tiempo y el enfoque en el desarrollo del prototipo funcional, las estrategias de depuración y pruebas manuales iterativas han sido fundamentales para asegurar la calidad y funcionalidad del sistema entregado.

Diseño de casos de prueba automatizados

Con Katalon Studio, se crearon un conjunto de casos de prueba para validar los flujos de trabajo más críticos. Estas pruebas se pueden clasificar como **pruebas de regresión funcional** y **pruebas de aceptación del sistema**. A continuación, se describen algunos ejemplos representativos:

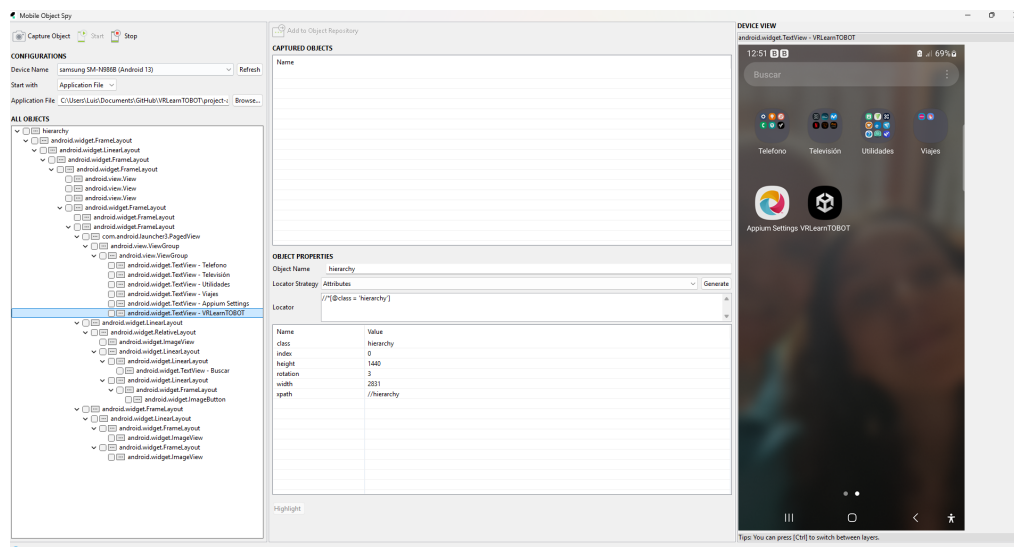


Figura D.1: Inicialización pruebas en dispositivo Android

1. **Caso de prueba TP-01: Ejecución exitosa de un programa simple**
 - **Objetivo:** Validar que un programa sintácticamente correcto se ejecuta completamente y el robot realiza la acción esperada.

■ Pasos Automatizados:

- a) Iniciar la aplicación y navegar al menú de selección de desafíos.
- b) Hacer clic en el Desafío de Movimiento Básico.
- c) En la escena de programación, introducir el comando mover 10 pasos en el campo de texto y añadirlo al programa.
- d) Hacer clic en el botón ejecutar.
- e) **Verificación:** Comprobar que (a) la simulación se inicia, (b) el robot avanza, y (c) al finalizar, el texto de salida muestra un mensaje de ¡Ejecución Completada!.



Figura D.2: Prueba en dispositivo Android

2. Caso de prueba TP-02: Detección de error de sintaxis

- a) **Objetivo:** Verificar que el parser detecta correctamente un comando inválido y proporciona feedback al usuario.
- b) **Pasos Automatizados:**
 - 1) ... (Iniciar y seleccionar desafío) ...

- 4) **Verificación:** Comprobar que (a) el robot se mueve repetidamente pero (b) la simulación se detiene antes de completar las 4 iteraciones y (c) el texto de salida muestra un mensaje de ¡Colisión!.

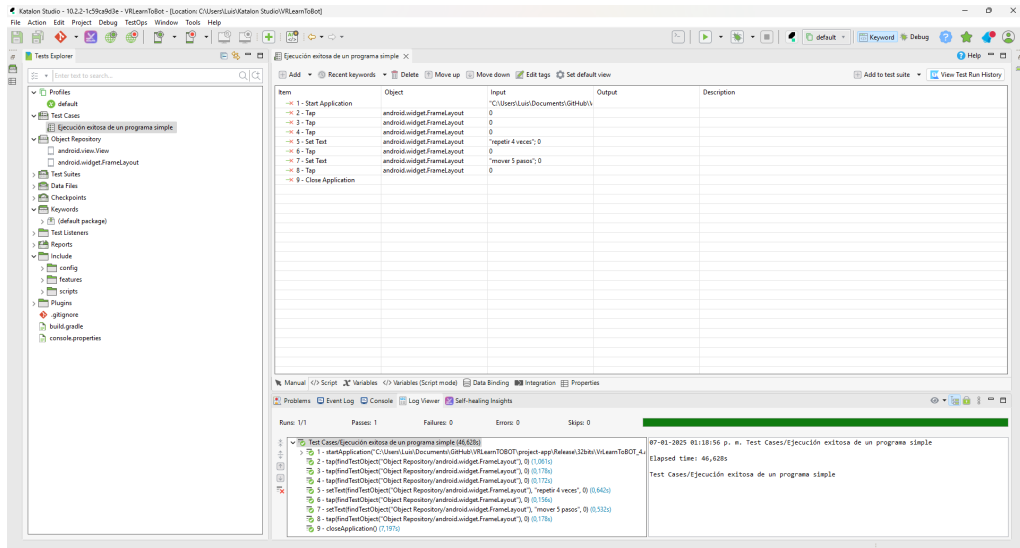


Figura D.4: Ejecución exitosa caso de prueba

El uso de Katalon Studio para la automatización de estas pruebas ha sido fundamental para garantizar la estabilidad del prototipo a lo largo de los ciclos de desarrollo, permitiendo detectar regresiones de forma temprana y eficiente.

Apéndice *E*

Documentación de Usuario

E.1. Introducción

Bienvenido al Laboratorio Virtual de Programación y Robótica VRLearn-Tobot. Esta guía está diseñada para ayudarte a instalar y utilizar todas las funcionalidades de esta herramienta educativa. El objetivo principal de este laboratorio es proporcionar un entorno accesible e interactivo donde puedas aprender los fundamentos de la programación mediante un **lenguaje de comandos textual o programación por bloques**, y observar cómo tus programas controlan un robot virtual en un **entorno 3D basado en desafíos**.

Este manual te guiará a través de los requisitos del sistema, el proceso de instalación en un dispositivo Android (Tablet), y el uso paso a paso de las características principales, desde la creación de tu primer programa hasta la simulación del comportamiento del robot.

E.2. Requisitos del sistema y del usuario

Requisitos de hardware y software del dispositivo

Para asegurar un correcto funcionamiento del Laboratorio Virtual, tu dispositivo (preferentemente una Tablet Android) debe cumplir con los siguientes requisitos mínimos:

- **Sistema operativo:** Android (versión recomendada: Android 8.0 Oreo o superior. Compatible con versiones anteriores con posible impacto en rendimiento).

- **Procesador:** CPU compatible con arquitectura ARM (ARMv7 con NEON o ARM64).
- **Memoria RAM:** Se recomienda un mínimo de 2GB de RAM para una experiencia fluida.
- **Almacenamiento interno:** Espacio libre suficiente para la instalación de la aplicación (aproximadamente 40 MB).
- **Pantalla:** Orientado para tablets, se recomienda una pantalla con resolución y tamaño adecuados para la interacción con bloques.

Permisos requeridos por la aplicación

Al instalar y/o ejecutar la aplicación por primera vez, podría solicitar los siguientes permisos:

- **Almacenamiento:** Para guardar y cargar tus proyectos de bloques. En el prototipo actual, se utiliza almacenamiento interno de la aplicación (`PlayerPrefs`), por lo que este permiso explícito podría no ser necesario.

La aplicación no requiere acceso a internet para su funcionamiento básico (programación y simulación local).

E.3. Instalación de la aplicación en dispositivo Android

El Laboratorio Virtual de Programación y Robótica se distribuye como un archivo `.apk` para su instalación directa en dispositivos Android. Sigue estos pasos:

1. **Obtener el archivo `.apk`:** Descarga el archivo `VrLearnToBOT.apk` desde la ubicación <https://github.com/LuisIgnaciodeLunaGomez/VRLearnTOBOT/tree/master/app/Release/64bits>.
2. **Habilitar fuentes desconocidas:**
 - En tu dispositivo Android, ve a **Ajustes > Seguridad** (la ruta exacta puede variar según el fabricante y la versión de Android).

- Busca y activa la opción que permite instalar aplicaciones de **fuentes desconocidas** o **instalar aplicaciones desconocidas**. Es posible que necesites conceder este permiso específicamente a tu gestor de archivos o navegador desde el cual intentarás abrir el APK.
- *Advertencia de Seguridad:* Este paso es necesario para instalar APKs fuera de Google Play Store. Se recomienda desactivar esta opción después de instalar la aplicación si no la necesitas para otras instalaciones.

3. Instalar el .apk:

- Navega hasta la ubicación donde descargaste el archivo .apk utilizando un gestor de archivos.
- Pulsa sobre el archivo VrLearnToBOT.apk.
- El sistema te preguntará si deseas instalar la aplicación. Confirma la instalación.
- Espera a que el proceso de instalación finalice.

4. **Abrir la aplicación:** Una vez instalada, encontrarás el icono del Laboratorio Virtual en tu lista de aplicaciones. Púlsalo para iniciar.

E.4. Manual de usuario del prototipo código escrito

Esta sección te guiará a través de las funcionalidades principales del prototipo del laboratorio virtual.

NOTA: Para volver hacia atrás en cualquier pantalla solo debes pulsar en el logotipo de la aplicación y te llevará a la pantalla anterior.

Paso 1: Seleccionar un desafío

Al iniciar la aplicación, serás recibido por el **Menú principal**, donde podrás elegir tu misión.



Figura E.1: Vista del menú principal .

1. Tras seleccionar código escrito se te mostrará una lista de **tarjetas de desafío** disponibles. Cada tarjeta contiene el título y una breve descripción del objetivo.
2. Pulsa sobre la tarjeta del desafío que desees resolver (ej. secuenciación).
3. Una vez seleccionado, el sistema te llevará automáticamente al entorno de programación, con el desafío ya cargado.



Figura E.2: Vista del menú principal para la selección de desafíos.

Paso 2: La interfaz de programación (IDE)

Una vez dentro, te encontrarás en el Entorno de Desarrollo Integrado (IDE). La interfaz se divide en las siguientes áreas clave (ver Figura E.3):

- **Panel de instrucciones del desafío (arriba):** Muestra el título y la descripción detallada del objetivo que debes cumplir. ¡Léelo con atención!
- **Área de programa:** Aquí es donde se visualizará tu programa, línea por línea, a medida que añades comandos.
- **Panel de comandos:** Es tu zona de trabajo principal. Contiene un campo de texto para escribir los comandos del robot.
- **Panel de salida y feedback :** Muestra mensajes importantes, como errores de sintaxis en tu código, el estado de la ejecución, o mensajes de éxito y colisión.



Figura E.3: Vista general de la Interfaz de Programación (IDE).

Paso 3: Construir tu programa

Tu misión es escribir la secuencia correcta de comandos para que el robot cumpla el objetivo del desafío.

1. **Escribir un comando:** En el panel de comandos, pulsa sobre el **campo de texto** para activar el teclado. Escribe tu primer comando. El lenguaje es simple y directo. Ejemplos de comandos válidos:

- mover 10 pasos
 - girar derecha
 - repetir 4 veces
2. **Añadir el comando al programa:** Una vez escrito el comando, pulsa el botón **+**. El comando se añadirá como un nuevo bloque de texto en el area de programa a la izquierda.
 3. **Construir la secuencia:** Repite el proceso para añadir todos los comandos necesarios en el orden correcto. Si te equivocas, puedes usar el botón **Borrar** para limpiar todo el programa y empezar de nuevo.
 4. **Manejo de bucles:** Para crear un bucle, primero escribe el comando repetir N veces y añádelo. Luego, añade todos los comandos que irán dentro del bucle. Finalmente, añade el comando fin_repetir para cerrar el bloque.



Figura E.4: Ejemplo de un programa construido con comandos textuales.

Paso 4: Ejecutar y simular el programa

1. **Iniciar simulación:** Cuando creas que tu programa es correcto, pulsa el botón **ejecutar**.
2. **Verificación del código:** El sistema analizará tu código. Si encuentra un error de sintaxis, aparecerá un mensaje en rojo en el panel de salida. ¡Deberás corregir tu código y volver a intentarlo!

3. **Observar al robot:** Si el código es correcto, la simulación comenzará. Observa cómo el robot ejecuta tus comandos en el entorno 3D. Un cronómetro medirá el tiempo de ejecución.
4. **Resultado del desafío:** La ejecución puede terminar de varias maneras:
 - **Éxito:** Si el robot completa el objetivo, recibirás un mensaje de felicitación.
 - **Colisión:** Si el robot choca con un muro, la ejecución se detendrá y verás un mensaje de colisión.

Cambio de cámara

Durante la simulación, puedes pulsar el botón de **cambio de cámara** para obtener diferentes perspectivas de la acción del robot, lo que te ayudará a entender mejor su comportamiento y a depurar tu programa.



Figura E.5: Simulación con cámara trasera.

E.5. Manual de usuario del prototipo programación por bloques

Esta sección te guiará a través de las funcionalidades principales del prototipo del laboratorio virtual.

NOTA: Para volver hacia atrás en cualquier pantalla solo debes pulsar en el logotipo de la aplicación y te llevará a la pantalla anterior.

Paso 1: Seleccionar itinerario

Si en el menú principal has seleccionado la opción bloques de colores , serás recibido por el **Menú principal**, donde podrás elegir entre programación libre o seleccionar un desafío de programación por bloques.



Figura E.6: Vista del menú principal bloques de colores.

Paso 2: Seleccionar desafío

Si has seleccionado desafío el sistema te ofrecerá distintos desafíos para que aprendas a programar y resolver distintos retos.



Figura E.7: Vista del menú de desafíos.

Paso 3: Descripción del desafío

Una vez has seleccionado un desafío se abrirá una pantalla con un video de demostración de este y algunas instrucciones. Si presionas en iniciar desafío te llevará a la pantalla de programación.

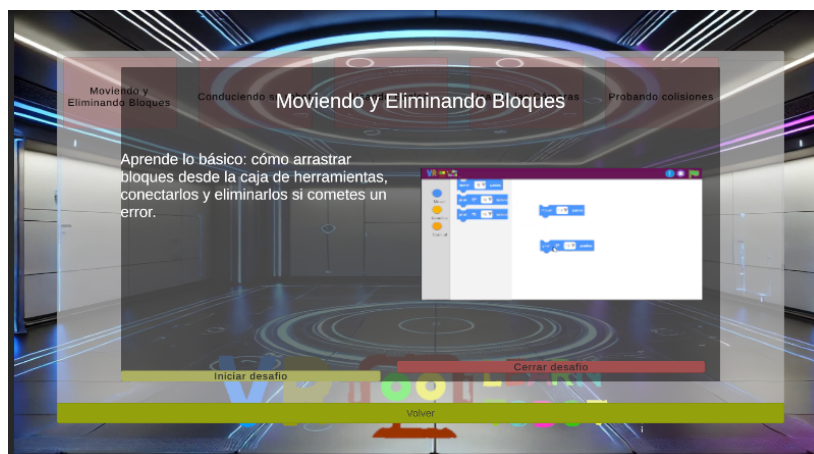


Figura E.8: Vista de la descripción del desafío.

Paso 4: Iniciar desafío

Has pulsado en iniciar el desafío propuesto por lo que estarás en la pantalla de programación por bloques donde deberás de hacer el programa para que el robot se mueva.

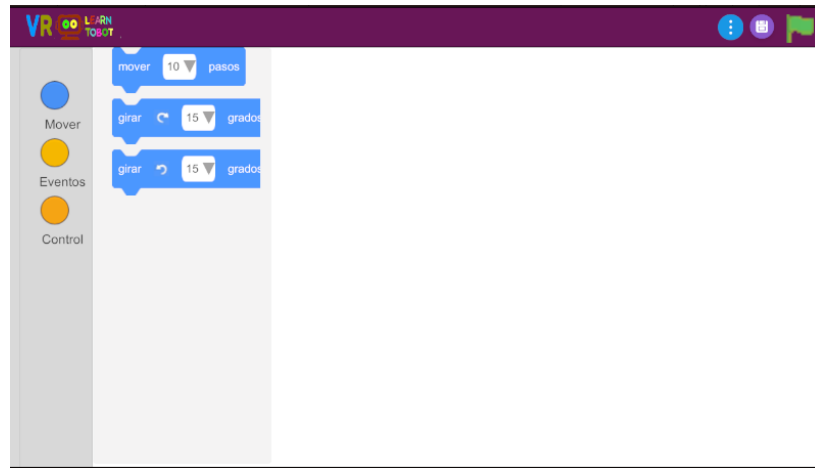


Figura E.9: Vista de la pantalla de programación por bloques.

Paso 5: Ejecutar desafío

Una vez has programado un desafío pulsa en la bandera verde para que se abra la zona de simulación donde el robot debe de ejecutar las acciones que le has indicado.



Figura E.10: Vista de la pantalla de simulación.

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

El presente anexo recoge una reflexión personal sobre cómo los principios de sostenibilidad se han integrado y abordado durante el desarrollo de este Trabajo Fin de Grado (TFG). Más allá de los aspectos puramente técnicos, el proyecto "Laboratorio Virtual de Programación y Robótica" ha ofrecido la oportunidad de explorar y aplicar una visión de la ingeniería informática como una disciplina al servicio del desarrollo sostenible. Se profundiza en las competencias de sostenibilidad adquiridas y su aplicación práctica, demostrando que las soluciones tecnológicas pueden ser catalizadores para un futuro más equitativo y responsable, en línea con las directrices de la CRUE [1]

F.2. Contribución del proyecto a los Objetivos de Desarrollo Sostenible (ODS)

El proyecto se alinea de manera fundamental con varios de los Objetivos de Desarrollo Sostenible de la ONU, especialmente en lo que respecta al acceso equitativo a la educación y la reducción de las desigualdades.

ODS 4: Educación de calidad

El **ODS 4: Educación de Calidad** busca garantizar una educación inclusiva, equitativa y de calidad, así como promover oportunidades de aprendizaje durante toda la vida para todos. El laboratorio virtual contribuye directamente a este objetivo al derribar barreras geográficas y económicas que limitan el acceso a la enseñanza de la programación y la robótica. La dependencia de laboratorios físicos bien equipados y kits de robótica específicos representa un obstáculo considerable para centros educativos con recursos limitados o en zonas remotas. Al ofrecer una alternativa de simulación en Unity, se democratiza el acceso a la educación STEAM (Ciencia, Tecnología, Ingeniería, Arte y Matemáticas), permitiendo a niños y adolescentes experimentar con la programación de robots sin necesidad de adquirir costosos materiales o infraestructura. Esto promueve la inclusión de un mayor número de estudiantes en disciplinas de alto valor añadido, fomentando el desarrollo del pensamiento computacional y las habilidades digitales cruciales para el siglo XXI, asegurando que el origen socioeconómico no sea un factor determinante en el acceso a la calidad educativa.

ODS 10: Reducción de las desigualdades

De forma inherente, el proyecto aborda el **ODS 10: Reducción de las desigualdades**, el cual aspira a reducir la desigualdad dentro y entre los países, facilitando la inclusión social, económica y política de todas las personas. El laboratorio virtual disminuye la brecha de acceso a la tecnología y al conocimiento especializado que a menudo existe en contextos de vulnerabilidad. Al priorizar el desarrollo para dispositivos Android tipo tablet, se garantiza que la plataforma sea accesible en una gama más amplia de hardware de bajo coste, extendiendo así la oportunidad de aprender programación y robótica a segmentos de la población que tradicionalmente quedarían excluidos. La gratuidad de las herramientas empleadas para su desarrollo (Unity Personal, Csharp, UBlockly de código abierto, Git/GitHub) reduce drásticamente el coste de implementación y replicación de esta solución, permitiendo que organizaciones educativas, comunidades y familias puedan acceder a recursos didácticos de vanguardia sin que la inversión económica sea un impedimento. Este enfoque minimiza las disparidades en las oportunidades de aprendizaje, contribuyendo a una sociedad más equitativa e inclusiva.

F.3. Competencias de sostenibilidad adquiridas y aplicadas

Durante el desarrollo de este TFG, he tenido la oportunidad de aplicar y fortalecer diversas competencias clave relacionadas con la sostenibilidad, trascendiendo el ámbito técnico y proyectándose hacia un impacto social y medioambiental.

Pensamiento crítico y sistémico

El desafío de crear un laboratorio virtual que sea accesible y educativo ha requerido un profundo pensamiento crítico y sistémico. Comprender las interconexiones entre la falta de recursos, las desigualdades educativas y la solución tecnológica implicaba analizar cómo cada componente del diseño (desde la elección de Unity y UBlockly hasta la simplificación del modelo de robot LEGO WeDo 2.0) impactaría en la accesibilidad global y en la viabilidad a largo plazo. Se ha procurado optimizar los algoritmos de simulación para un menor consumo de recursos, y el diseño de la interfaz para la usabilidad en pantallas pequeñas, evidenciando una perspectiva integral sobre cómo la tecnología se inserta en un contexto social y económico.

Diseño orientado al impacto social y a largo plazo

La reflexión sobre la sostenibilidad ha influido significativamente en las decisiones de diseño. En lugar de desarrollar una solución ad-hoc, se ha priorizado la creación de un sistema extensible y modular que pueda evolucionar. La elección de UBlockly y la arquitectura MVC responden a la necesidad de construir una base que pueda adaptarse a nuevos tipos de bloques, modelos de robot y plataformas (como futuras versiones web o de realidad virtual) sin tener que reinventar la rueda, prolongando la vida útil del software y maximizando su potencial impacto educativo a largo plazo. Esta visión a largo plazo minimiza el despilfarro de recursos de desarrollo y facilita la resiliencia del proyecto ante futuros cambios tecnológicos o necesidades pedagógicas.

Gestión ética de la información y responsabilidad profesional

La utilización de herramientas de código abierto como UBlockly y la contribución implícita a la comunidad de conocimiento libre forman parte

de una gestión responsable y ética. Se ha prestado atención a la calidad y claridad del código, así como a la documentación técnica, para asegurar que el proyecto pueda ser mantenido y extendido por otros desarrolladores, fomentando la colaboración y el intercambio de conocimientos. Además, la conciencia sobre la importancia de la privacidad de los usuarios (aunque este prototipo no gestione datos sensibles, la mentalidad de diseñar para la seguridad futura es clave) y la integridad del proceso de desarrollo han sido principios rectores que han consolidado mi comprensión de la responsabilidad que implica el diseño de sistemas informáticos.

F.4. Reflexión final

El desarrollo de este TFG ha sido una experiencia transformadora que ha trascendido la mera aplicación de conocimientos técnicos en programación. Me ha permitido conectar la teoría de la ingeniería informática con desafíos sociales concretos y con los principios universales de la sostenibilidad. La convicción de que la tecnología bien diseñada, con un propósito claro y un enfoque ético, tiene un potencial inmenso para contribuir a un mundo más justo y equitativo se ha reforzado enormemente. Este proyecto es una muestra de cómo las competencias técnicas y de sostenibilidad se entrelazan para dar vida a soluciones innovadoras que abren nuevas oportunidades educativas y contribuyen al bienestar global. Este trabajo no es solo un ejercicio académico, sino el inicio de una vocación por construir un futuro donde la tecnología sirva como un verdadero motor de cambio sostenible.

Bibliografía

- [1] CRUE Universidades Españolas. Directrices para la introducción de la sostenibilidad en el currículo universitario. https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf, 2012. [En línea; consultado el 30-mayo-2025].
- [2] Zube Inc. Zube - project development managemet for agile development. <https://zube.io>, 2025.